ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SCUOLA DI INGEGNERIA E ARCHITETTURA

Dipartimento di Informatica – Scienza e Ingegneria

Corso di Laurea in Ingegneria Informatica

BINARY SIMILARITY: ANALISI COMPARATIVA DI APPROCCI TRADIZIONALI E METODI BASATI SU MACHINE LEARNING

Relatore:
Prof. Dr. Marco Prandini

Correlatore:
Dr. Antonio Iacobelli

Presentata da:
Alessandra Tolomelli

Sessione di Ottobre 2025
Anno Accademico 2024/2025

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SCUOLA DI INGEGNERIA E ARCHITETTURA

Dipartimento di Informatica – Scienza e Ingegneria

Corso di Laurea in Ingegneria Informatica

BINARY SIMILARITY: ANALISI COMPARATIVA DI APPROCCI TRADIZIONALI E METODI BASATI SU MACHINE LEARNING

Relatore:
Prof. Dr. Marco Prandini

Correlatore:
Dr. Antonio Iacobelli

Presentata da:
Alessandra Tolomelli

Sessione di Ottobre 2025
Anno Accademico 2024/2025

Abstract

La crescente diffusione di software compilato per architetture eterogenee e l'intensificarsi delle minacce informatiche hanno reso la Binary Code Similarity Detection (BCSD) una disciplina centrale nell'ambito dell'analisi binaria. La capacità di confrontare frammenti di codice binario, anche in assenza del sorgente, è fondamentale per applicazioni quali il rilevamento di vulnerabilità e l'analisi di malware. Tuttavia, la BCSD è resa complessa da fattori come le ottimizzazioni del compilatore, le differenze tra Instruction Set Architectures (ISA) e le tecniche di offuscamento.

Questa tesi propone un'analisi comparativa tra approcci tradizionali e metodi basati su Machine Learning, con l'obiettivo di valutarne efficacia, limiti e applicabilità. Vengono esaminati contributi recenti che impiegano modelli linguistici e tecniche di embedding per rappresentare il codice binario in spazi semantici, superando le barriere sintattiche e architetturali. Particolare attenzione è rivolta all'uso di rappresentazioni intermedie e meccanismi di attenzione contestuale, che permettono ai modelli di cogliere relazioni profonde tra istruzioni e blocchi di codice.

Il confronto tra gli approcci evidenzia come le tecniche basate su apprendimento automatico offrano maggiore flessibilità e capacità di generalizzazione, pur presentando sfide legate alla disponibilità di dati e all'efficienza computazionale. Le soluzioni tradizionali, invece, mantengono un vantaggio in termini di interpretabilità e robustezza.

Indice

Abstract **i**

1 Introduzione **1**

2 Background **3**

2.1 Origini ed evoluzione della Binary Code Similarity 3

2.1.1 Primo periodo 4

2.1.2 Secondo periodo 5

2.2 Binary Code Analysis e Reverse Engineering 6

2.2.1 IDA Pro 8

2.2.2 Codice macchina vs codice assembly 8

2.2.3 Instruction Set Architecture 9

2.2.4 Control Flow Graph e Blocchi base 10

2.3 Tecniche di embedding e rappresentazione semantica 11

2.3.1 Word2Vec 12

2.3.2 FastText 12

2.3.3 Asm2Vec 13

2.4 Modelli di Machine Learning e deep learning 14

2.4.1 Reti Neurali 14

2.4.2 Transformer 15

2.4.3 Graph Neural Network 18

3 Analisi di approcci selezionati alla Binary Similarity **21**

3.1 Unsupervised Binary Code Translation with Application to Code Similarity Detection and Vulnerability Discovery 21
3.1.1 Metodologia 22
3.1.2 Risultati ottenuti 24
3.1.3 Applicazioni pratiche 26
3.1.4 Conclusioni 27
3.2 jTrans: Jump-Aware Transformer for Binary Code Similarity Detection 28
3.2.1 Metodologia 29
3.2.2 BinaryCorp 33
3.2.3 Risultati ottenuti 34
3.2.4 Applicazioni pratiche 38
3.2.5 Conclusioni 39
3.3 Binary Similarity Detection using Machine Learning 40
3.3.1 Metodologia 40
3.3.2 Risultati ottenuti 45
3.3.3 Conclusioni 46
3.4 Statistical Similarity of Binaries 46
3.4.1 Metodologia utilizzata 47
3.4.2 Risultati ottenuti 51
3.4.3 Conclusioni 53
3.5 Cross-Architecture bugs search in binary executables 54
3.5.1 Metodologia 55
3.5.2 Risultati ottenuti 59
3.5.3 Conclusioni 60

4 Confronto tra gli approcci analizzati 63
4.1 Confronto tra metodi basati su Machine Learning 63
4.2 Confronto tra metodi tradizionali 65
4.3 Confronto tra approcci ML e metodi tradizionali 67

5 Conclusioni e sviluppi futuri 71

Bibliografia **73**

Ringraziamenti **79**

Capitolo 1

Introduzione

Negli ultimi anni, la crescente diffusione di software compilato per architetture eterogenee e l'aumento delle minacce informatiche hanno reso la Binary Code Similarity Detection (BCSD) una tematica centrale nell'ambito dell'analisi binaria.
La capacità di identificare somiglianze tra frammenti di codice binario, anche in assenza del codice sorgente, è fondamentale per numerose applicazioni, tra cui il rilevamento di vulnerabilità, l'analisi di malware e la classificazione automatica di eseguibili. In questo contesto, la ricerca ha progressivamente abbandonato approcci puramente sintattici, orientandosi verso tecniche capaci di cogliere relazioni semantiche profonde, spesso supportate da modelli di Machine Learning e deep learning.
La presente tesi si propone di analizzare in modo comparativo i principali approcci alla binary similarity, distinguendo tra metodi tradizionali e tecniche basate su apprendimento automatico. L'obiettivo è quello di valutare i punti di forza e le limitazioni di ciascun modello, offrendo una panoramica critica e strutturata delle soluzioni esistenti.
Il Capitolo 2 introduce contesto teorico e storico della disciplina, presentando i concetti fondamentali dell'analisi binaria e del reverse engineering. Vengono inoltre descritti i principali modelli di rappresentazione semantica e le architetture di Machine Learning impiegate nell'ambito della BCSD.

Nel Capitolo 3 vengono analizzati in dettaglio cinque contributi scientifici selezionati, tre dei quali basati su modelli di apprendimento automatico e due fondati su tecniche tradizionali. Per ciascun approccio vengono esaminati la metodologia, i risultati sperimentali, le applicazioni pratiche e le prospettive di sviluppo.
Il Capitolo 4 è dedicato al confronto tra gli approcci analizzati. Viene proposta una riflessione critica sui limiti metodologici e la sezione finale del capitolo mette a confronto le prestazioni dei metodi basati su Machine Learning con quelle degli approcci tradizionali, evidenziando le differenze in termini di accuratezza, interpretabilità e applicabilità.
Infine, il Capitolo 5 raccoglie le conclusioni del lavoro, sintetizzando i risultati emersi e delineando le principali direzioni di ricerca futura.

Capitolo 2

Background

2.1 Origini ed evoluzione della Binary Code Similarity

La Binary Code Similarity nasce con l'obiettivo di generare una patch che catturi le differenze tra due versioni consecutive di un programma. Gli strumenti di confronto testuale esistono dagli anni '70 e sono stati integrati nei primi *Version Control System* (VCS): strumenti software che tracciano e gestiscono le modifiche apportate ai file nel tempo, facilitando la collaborazione e il controllo delle versioni. Tuttavia, la crescente diffusione di reti wireless e le risorse limitate di alcuni dispositivi hanno stimolato l'interesse verso tecniche che aumentassero l'efficienza, trasmettendo solo una piccola patch contenente le differenze tra due versioni di un file binario invece dell'intero file.
I primi studi sulla binary code similarity risalgono al 1999, con lo sviluppo di un prototipo denominato Exediff [1], progettato per generare patch di eseguibili DEC Alpha. Le analisi mostrarono che molte differenze tra due versioni binarie dello stesso programma non derivavano da modifiche del codice sorgente, ma dal processo di compilazione. Un esempio tipico è l'allocazione dei registri, che può cambiare a ogni ricompilazione. L'approccio proposto consisteva nel ricostruire tali cambiamenti secondari al momento dell'appli-

cazione della patch, evitando di includerli direttamente in essa e riducendone così le dimensioni. Exediff può quindi essere considerato il primo metodo noto a focalizzarsi sul calcolo della similarità tra codici binari, sfruttando la struttura del programma ottenuta tramite il disassemblaggio dei byte grezzi in istruzioni assembly [2].
Sempre nel 1999, venne presentato anche BMAT [3], uno strumento che allineava due versioni di una libreria eseguibile DLL di Windows per trasferire le informazioni di profilazione dalla versione più vecchia, ampiamente profilata, a quella più recente, riducendo così la necessità di una nuova profilazione. Questo approccio, a differenza di Exediff che si limitava al confronto tra sequenze di istruzioni, è il primo a comparare blocchi base.
Inizialmente, venivano abbinate le funzioni nei due eseguibili e, successivamente, confrontati i blocchi simili all'interno di ciascuna funzione abbinata. Il modello utilizzava una tecnica di hashing per confrontare i blocchi, rimuovendo le informazioni di rilocazione per gestire le modifiche ai puntatori, ma era sensibile all'ordine.

2.1.1 Primo periodo

Dopo i lavori iniziati su Exediff e BMAT, nel decennio successivo (2000-2009) sono stati identificati sette approcci alla similarità, alcuni di questi altamente influenti poichè estendono la similarità del codice binario da un confronto puramente sintattico a uno che include anche aspetti semantici. Il campo d'applicazione viene ampliato dal semplice diffing del codice binario all'inclusione del clustering e della ricerca, e la similarità del codice binario applicata all'analisi di malware.
Nel 2004 venne proposto un approccio di diffing del codice binario basato su grafi, focalizzato sulle proprietà strutturali del codice, costruendo euristicamente un isomorfismo tra grafi delle chiamate per allineare le funzioni in due versioni dello stesso programma binario. Questo è il primo approccio in grado di gestire il riordinamento delle istruzioni all'interno di una funzione, introdotto da alcune ottimizzazioni del compilatore. Un lavoro successivo

estese l'approccio includendo anche il confronto tra blocchi base all'interno delle funzioni abbinate. Questi due lavori sono alla base del noto plugin BinDiff [4] per il disassemblatore IDA [5].
Nel 2005, venne proposta una tecnica di colorazione dei grafi [6] per rilevare varianti polimorfiche di malware. Questo è il primo approccio che ha effettuato un confronto semantico e una comparazione Many-to-Many. Le istruzioni sono state classificate con funzionalità simili in 14 classi semantiche e, successivamente, il control flow graph (CFG) è stato colorato utilizzando queste classi. La colorazione del grafo è robusta contro offuscamenti sintattici come l'inserimento di istruzioni inutili, il riordinamento delle istruzioni e la sostituzione delle istruzioni.
Nel 2008, venne poi proposto BinHunt [7] per identificare differenze semantiche tra due versioni dello stesso programma. Questo è il primo approccio che verifica l'equivalenza del codice, intesa come la capacità di determinare se due frammenti di programma, pur avendo rappresentazioni sintattiche differenti, presentano lo stesso comportamento semantico.
Nel 2009, invece, venne presentato Smit [8], un approccio che, dato un campione di malware, trova malware simili in un archivio. Smit è il primo approccio One-to-Many e di ricerca del codice binario. Indicizza i grafi delle chiamate dei malware in un database e utilizza la distanza di modifica tra grafi per trovare malware con grafi delle chiamate simili.

2.1.2 Secondo periodo

Nel periodo più recente, a partire dal 2010, si è registrato un notevole incremento di interesse verso la similarità del codice binario, con l'identificazione di ben 52 metodi. La ricerca si è concentrata principalmente su tecniche di confronto binario, con una crescente attenzione, a partire dal 2015, alla versione cross-architecture e, negli ultimi anni, all'impiego di metodi di apprendimento automatico.
Nel 2013, venne proposto Rendezvous [9], un motore di ricerca per codice binario che, dato il codice binario di una funzione di query, trova altre fun-

zioni in un archivio con proprietà sintattiche e strutturali simili. Ridurre la granularità della ricerca da interi programmi, analogamente a quanto fatto da Smit, a porzioni più piccole come le funzioni ha aperto la strada a molte applicazioni, tra cui il rilevamento di cloni e la ricerca di vulnerabilità.
La maggior parte degli approcci di ricerca del codice binario è orientata alla ricerca di bug. Questa applicazione è stata affrontata per la prima volta sul codice sorgente nel 2012 da Jang et al. [10].
Nel 2014 venne proposto Tracy [11], il primo approccio di ricerca binaria focalizzato sulla ricerca di bug. Tracy utilizzava il concetto di tracelets, ovvero percorsi di esecuzione all'interno di un control flow graph, per trovare funzioni simili a una funzione vulnerabile.
Nel 2015 venne presentato Multi-MH [12], il primo approccio di ricerca binaria cross-architecture. Multi-MH indicizzava le funzioni in base alla loro semantica input-output. Dato un codice compilato per una certa architettura (es. x86), Multi-MH poteva trovare funzioni simili compilate per altre architetture (es. MIPS). Questo problema ha rapidamente guadagnato interesse grazie alla diffusione dei dispositivi embedded.
Nel 2016 venne addestrata una rete neurale [13] per decidere se due funzioni fossero state compilate dallo stesso codice sorgente. Da allora, l'uso delle reti neurali è cresciuto rapidamente, con approcci come Word2vec (2013) [14], FastText (2016) [15], Gemini (2017) [16], αDiff (2018) [17], Asm2Vec (2019) [18], DeepBinDiff (2020) [19] e jTrans (2022) [20].

2.2 Binary Code Analysis e Reverse Engineering

L'analisi binaria consiste nell'analizzare i programmi a livello di codice macchina, in modo da identificarne la funzionalità e il comportamento; in particolare, quando il codice sorgente non è accessibile. Questo processo viene applicato in diversi scenari, tra cui l'identificazione di bug o vulnerabilità in software, l'applicazione di patch in ambienti di produzione e l'analisi delle

funzionalità di eseguibili malevoli.
Per raggiungere questi obiettivi, sono state sviluppate diverse tecniche di analisi binaria, che si suddividono in due categorie principali: *analisi statica* e *analisi dinamica*.
Nel primo caso, l'analisi avviene senza eseguire il programma, offrendo così maggiore flessibilità, poichè possono essere analizzate su qualsiasi architettura CPU. Questa fase non è semplice poichè alcune architetture hanno istruzioni di lunghezza variabile che rendono il processo complicato. Inoltre, il codice assembly non prevede una suddivisione netta tra parte di codice e parte di dati, il che significa che istruzioni e dati possono essere mescolati insieme, complicando ulteriormente il processo.
Al contrario, l'analisi dinamica richiede l'esecuzione del programma in un ambiente isolato, su un'architettura in grado di supportare quel formato binario. Sebbene meno versatile, siccome limitato al flusso seguito durante l'esecuzione del programma, questo metodo semplifica il processo di analisi consentendo l'osservazione in tempo reale del comportamento del binario [21].
La Binary Code Similarity Detection (BCSD) è fondamentale per calcolare la somiglianza tra due funzioni binarie e può essere applicata in tre diversi scenari:

- One-to-One (OO): il punteggio di similarità viene calcolato tra una funzione sorgente e una target;
- One-to-Many (OM): un insieme di funzioni target viene ordinato in base ai punteggi di similarità rispetto ad una funzione sorgente;
- Many-to-many (MM): un pool di funzioni viene suddiviso in gruppi in base alla loro similarità.

Con *funzione* si intende un insieme ordinato di istruzioni in programmi binari che sono compilate da una funzione nel codice sorgente. Perciò, una funzione possiede una specifica semantica e degli specifici vincoli interni. Più precisamente, essa possiede un Control Flow Graph (CFG) che rappresenta

le informazioni sul suo flusso di esecuzione.
Con *compito* della BCSD, invece, dato un sorgente f_q e in insieme di funzioni P, si intende l'atto di recuperare le k funzioni più simili $\{f_1, f_2, \ldots, f_k \mid f_i \in P\}$ classificate in base al punteggio di similarità.
L'obiettivo è quello di sviluppare una soluzione che sia in grado di calcolare i punteggi di similarità tra due funzioni, dove queste ultime derivano da sorgenti che sono uguali o simili in una certa misura.
Per ottenere una completa comprensione delle caratteristiche di un binario, è spesso necessario ricorrere al reverse engineering manuale. Nonostante sia una delle attività più complesse dell'analisi binaria, rimane il metodo più efficace per scoprire la logica interna del programma. Questo processo richiede l'uso di un disassembler per recuperare le istruzioni assembly, spesso affiancato da un decompiler per ottenere una versione a livello più alto del codice, e da un debugger per monitorare il comportamento del programma.

2.2.1 IDA Pro

Il panorama dei disassembler offre diverse opzioni, tra cui IDA Pro [5]. Probabilmente il disassembler più popolare, è ampiamente utilizzato per il reverse engineering: permette di analizzare file eseguibili di diversi sistemi operativi e processori. Lo scopo principale è capire il funzionamento interno di un programma, identificare vulnerabilità o analizzare malware.

2.2.2 Codice macchina vs codice assembly

La differenza principale tra codice macchina e codice assembly è che il codice macchina è costituito da binari che possono essere eseguiti direttamente da un computer, mentre il codice assembly è un linguaggio di basso livello che richiede un assembler per convertirlo nel codice macchina. Il codice assembly, quindi, è un linguaggio intermedio che si trova tra il linguaggio di alto livello e il codice macchina.

2.2.3 Instruction Set Architecture

Un'Instruction Set Architecture (ISA) [22] è una specifica che definisce l'insieme di tutti i codici binari (opcode) che rappresentano i comandi eseguibili nativamente da un particolare design di CPU. In particolare, essa costituisce il linguaggio macchina del processore e rappresenta l'insieme di istruzioni base che la CPU è in grado di eseguire.
L'ISA agisce come interfaccia cruciale tra l'hardware e il software, definendo sia le operazioni che il processore è in grado di eseguire, sia le modalità con cui tali operazioni vengono implementate. Essa fa parte del modello astratto di un computer e stabilisce il modo in cui la CPU viene controllata dal software.
A partire da questa base di istruzioni macchina, vengono sviluppati programmi utilizzando linguaggi di programmazione ad alto livello di astrazione, che poi vengono tradotti nelle istruzioni native dell'ISA attraverso processi di compilazione o interpretazione. Essa definisce i tipi di dati supportati, i registri disponibili, il modo in cui l'hardware gestisce la memoria principale, le funzionalità chiave, le istruzioni che un microprocessore può eseguire e il modello di input/output adottato dalle diverse implementazioni dell'ISA.
È possibile distinguere due macro-categorie di ISA: i RISC (Reduced Instruction Set Computer) e i CISC (Complex Instruction Set Computer).
Le ISA RISC tendono ad avere operazioni semplici e veloci, con un'ampia disponibilità di registri per memorizzare i risultati intermedi.
Invece, le ISA CISC mettono a disposizione del programmatore istruzioni più complesse, che talvolta replicano quelle dei linguaggi di alto livello.
Le ISA più utilizzate sono x86 e ARM. x86 è un'architettura CISC di Intel e AMD, che privilegia potenza e flessibilità, ma consuma più energia ed è più complessa. ARM è un'architettura RISC sviluppata da ARM Holdings, ottimizzata per l'efficienza energetica e adatta sia a dispositivi mobili che a supercomputer grazie alle recenti ottimizzazioni di prestazioni [23].

2.2.4 Control Flow Graph e Blocchi base

Una volta compreso il ruolo delle ISA e delle istruzioni, diventa fondamentale analizzare il modo in cui queste istruzioni si concatenano tra loro durante l'esecuzione del programma.
Infatti, non sempre il codice viene eseguito in maniera lineare e, la presenza di salti condizionati, chiamate a funzioni e cicli può rendere il flusso di esecuzione più complesso.
Per studiare questo comportamento, si utilizzano concetti come blocchi base e Control Flow Graph (CFG).
Un blocco base è una sequenza di istruzioni eseguite in maniera sequenziale senza interruzioni.
Un CFG [24] rappresenta il flusso di controllo tra questi blocchi, permettendo di visualizzare come il programma possa evolvere a seconda delle condizioni e dei salti. È un grafo orientato nel quale ogni nodo rappresenta un blocco base e ogni arco il flusso di controllo tra i blocchi.

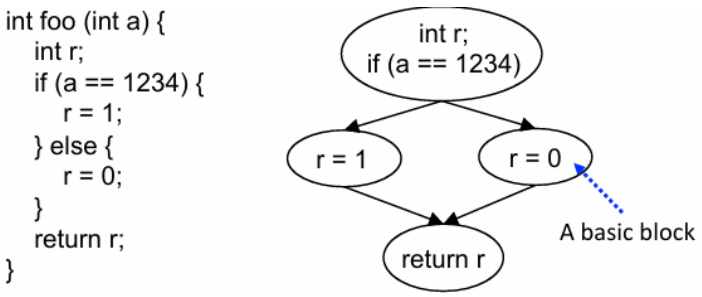

Figura 2.1: Control Flow Graph e blocco base [25].

Come mostrato nella Figura 2.1, dato un frammento di codice sorgente viene creato il suo corrispondente CFG dove ogni nodo è un blocco base. Allo stesso modo è anche possibile generare il corrispondente CFG di un frammento di codice binario.

2.3 Tecniche di embedding e rappresentazione semantica

Il *word embedding* permette di memorizzare le informazioni sia semantiche che sintattiche delle parole, partendo da un corpus non annotato e costruendo uno spazio vettoriale in cui i vettori delle parole sono più vicini se le parole occorrono negli stessi contesti linguistici, cioè se sono riconosciute come semanticamente più simili. Nell'elaborazione del linguaggio naturale indica un insieme di tecniche di modellazione in cui parole o frasi di un vocabolario vengono mappate in vettori di numeri reali. Concettualmente consiste in un'operazione matematica di immersione in conseguenza della quale uno spazio costituito da una dimensione per parola viene trasformato in uno spazio vettoriale continuo di dimensione molto inferiore. Esistono diversi metodi per generare la mappatura: tra questi, le reti neurali rappresentano una delle architetture più utilizzate per creare nuove tecniche di word embedding.
Assegnando valori numerici alle parole in base alle somiglianze semantiche, il word embedding consente ai modelli delle reti neurali di comprendere il contesto in modo più efficiente. Questo approccio riduce la complessità di elaborazione e migliora le prestazioni dei modelli, preservando le informazioni semantiche.
Dopo aver introdotto il concetto generale di word embedding, è utile presentare alcuni dei modelli più diffusi ed estesi, che verranno descritti nei paragrafi successivi. È interessante notare come molti modelli di word embedding, originariamente sviluppati per i linguaggi testuali, siano stati successivamente adattati e applicati a problemi di binary similarity.
Word2Vec [14] e FastText [15], ad esempio, hanno trovato applicazione in domini diversi da quello testuale. In altri casi, invece, partendo da questi modelli consolidati sono stati sviluppati approcci analoghi pensati specificamente per l'analisi di file binari, come Asm2Vec [18], che adatta i principi del word embedding al contesto dell'analisi del codice assembly e dei file eseguibili.

2.3.1 Word2Vec

Word2Vec [14] è un insieme di modelli che sono utilizzati per produrre word embedding. Si tratta di una rete neurale artificiale progettata per elaborare il linguaggio naturale. L'algoritmo richiede come input un grande corpus di testo, da cui apprende le relazioni tra le parole. Questo corpus viene trasformato in coppie di parole che appaiono vicine l'una all'altra nel testo. Viene utilizzata una rete neurale shallow, ovvero con un solo hidden layer, per apprendere i vettori di parole. Ciascuna parola nel vocabolario viene inizialmente rappresentata da un vettore di numeri casuali. Questi vettori vengono poi aggiornati via via che la rete neurale apprende dalle coppie di parole.
L'obiettivo dell'algoritmo è modificare i vettori di parole in modo tale che parole con significati simili finiscano per avere vettori simili. Questo si ottiene attraverso il processo di apprendimento della rete neurale, che cerca di predire una parola data la parola adiacente o viceversa.
Una volta addestrato, Word2Vec può essere usato per calcolare la similarità tra parole o risolvere analogie tra le altre applicazioni.
È importante notare che l'algoritmo non ha una comprensione innata del linguaggio: apprende solo le relazioni tra parole basate sui dati di addestramento. Quindi, la qualità dei vettori di parole dipenderà in gran parte dalla qualità e dalla quantità del corpus di testo usato per l'addestramento.

2.3.2 FastText

Le tecniche di word embedding come Word2Vec forniscono rappresentazioni vettoriali distinte per le parole presenti nel vocabolario: questo porta a tralasciare la struttura interna della lingua. Questa limitazione diventa significativa per le lingue morfologicamente ricche, siccome viene ignorata la relazione sintattica tra le parole. Poiché molte formazioni delle parole seguono regole precise nelle lingue morfologicamente ricche, è possibile migliorare le rappresentazioni vettoriali di queste lingue usando informazioni a livello

di carattere.
Per migliorare la rappresentazione vettoriale nelle lingue morfologicamente ricche, FastText [15] è un'estensione del modello Word2Vec e fornisce embedding per i character n-grams, ossia sequenze contigue di n caratteri estratte da una parola o da un testo, rappresentando le parole come la media di questi embedding.
Questo algoritmo è anche in grado di gestire le parole Out Of Vocabulary (OOV), cioè parole non presenti al momento dell'addestramento dei dati. Poiché FastText fornisce embedding per i character n-grams, può generare embedding anche per le parole OOV. Quando una parola OOV compare, FastText fornisce il suo embedding calcolando quello dei suoi character n-grams.

2.3.3 Asm2Vec

Asm2Vec [18] è un algoritmo di embedding progettato per rappresentare funzioni o blocchi di codice binario come vettori numerici in uno spazio continuo, simile a quanto Word2Vec fa per le parole nel linguaggio naturale.
Il modello tratta le istruzioni assembly come token, creando rappresentazioni vettoriali che catturano le relazioni semantiche tra le istruzioni. In questo modo, due funzioni che svolgono operazioni simili, anche se compilate in modi diversi o ottimizzate diversamente, risultano vicine nello spazio embedding.
Asm2Vec viene utilizzato principalmente in ambito di reverse engineering, rilevamento malware, analisi binaria e code similarity, permettendo di confrontare funzioni e rilevare pattern simili tra binari senza dipendere dalla forma esatta del codice sorgente o dai registri utilizzati. Questo approccio facilita anche l'analisi di malware e la scoperta di vulnerabilità, rendendo più robusta l'identificazione automatica di comportamenti simili tra programmi.

2.4 Modelli di Machine Learning e deep learning

Dopo aver descritto le tecniche di embedding e di rappresentazione semantica, è naturale analizzare i modelli di Machine Learning e deep learning che fanno uso di tali rappresentazioni. Gli embedding, infatti, forniscono una forma compatta e significativa dei dati (testo, codice o istruzioni binarie) che può essere utilizzata come input per modelli predittivi o di classificazione.
Nel corso degli anni sono emersi diversi approcci per sfruttare queste rappresentazioni: i modelli basati su reti neurali, le quali sono stati inizialmente applicati per la traduzione di una lingua ad un altra e poi successivamente sono stati applicati anche al codice; i modelli basati su Transformer [26] hanno rivoluzionato il campo grazie al meccanismo di attenzione e alla capacità di apprendere relazioni contestuali profonde; infine, le Graph Neural Network (GNN) [27] si sono affermate come strumento fondamentale per rappresentare e analizzare la struttura dei programmi attraverso grafi di flusso e alberi sintattici.
Nei paragrafi seguenti verranno quindi descritti questi tre approcci, evidenziandone i principi di funzionamento e le principali applicazioni nell'ambito dell'analisi del codice.

2.4.1 Reti Neurali

Le reti neurali sono modelli matematici e computazionali ispirati al funzionamento del sistema nervoso biologico, che simulano la struttura dei neuroni e le modalità con cui questi elaborano e trasmettono informazioni. Una rete neurale è composta da nodi, ovvero neuroni artificiali, organizzati in strati: uno strato di input, uno o più strati nascosti, e uno strato di output. Ogni nodo riceve segnali dai nodi del livello precedente, li elabora tramite una funzione di attivazione, e trasmette il risultato ai nodi successivi, modulando il flusso di informazione attraverso pesi sinaptici.
Durante il processo di apprendimento, la rete regola iterativamente questi

pesi per minimizzare l'errore tra le predizioni della rete e i valori desiderati, tipicamente usando algoritmi come la *backpropagation* [28].
Questa struttura permette alle reti neurali di generalizzare pattern complessi dai dati e di adattarsi a nuove situazioni, rendendole strumenti fondamentali in applicazioni come la classificazione di immagini, l'elaborazione del linguaggio naturale, la predizione di serie temporali e l'analisi di segnali.
Architetture più avanzate, come le Graph Neural Networks (GNN) [27], estendono questo paradigma per trattare strutture a grafo, sfruttando meccanismi di propagazione dei messaggi tra nodi per catturare relazioni strutturali e informazioni topologiche [29].

2.4.2 Transformer

Un Transformer [26] è un modello di apprendimento automatico che adotta il meccanismo della auto-attenzione, pesando in modo differente la significatività di ogni parte dei dati in ingresso. Inizialmente, l'architettura era utilizzata principalmente nelle branche dell'elaborazione del linguaggio naturale e della visione artificiale. Negli ultimi anni, però, si è dimostrata particolarmente efficace anche nel campo della binary similarity, consentendo di rappresentare e confrontare programmi binari in maniera più accurata e scalabile. Questo è possibile grazie alla capacità dei Transformer di apprendere relazioni a lungo raggio tra istruzioni e blocchi di codice.
I Transformer sono progettati per processare dati sequenziali con l'applicazione alla traduzione e la sintetizzazione di testi. Sono, inoltre, in grado di elaborare l'intero insieme di dati d'ingresso contemporaneamente. Il meccanismo dell'attenzione fornisce il contesto per ogni posizione nella sequenza di ingresso.

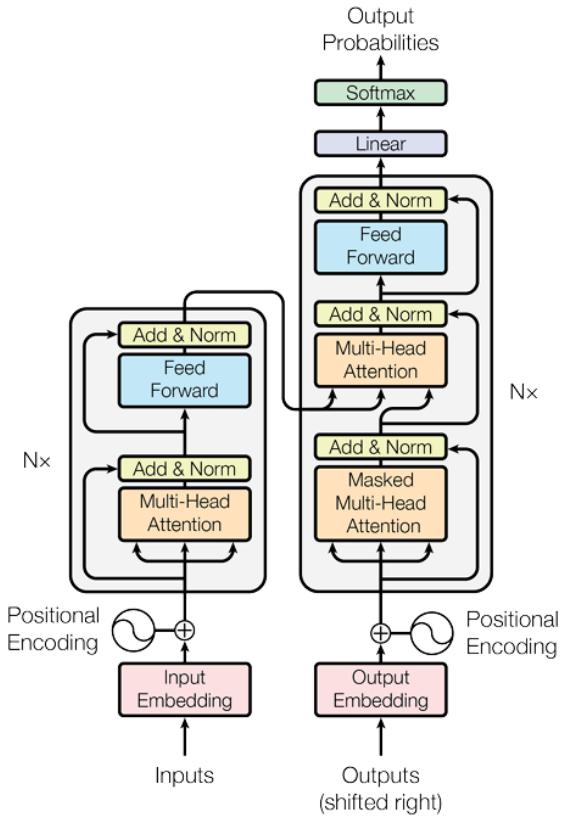

Figura 2.2: Architettura del Transformer [26].

L'architettura Transformer, mostrata in Figura 2.2, è composta da cinque parti principali: tokenizzazione, embedding, codifica posizionale, blocchi Transformer e strato softmax.
La quarta parte, il blocco Transformer, è la più complessa.
Questi blocchi possono essere concatenati, e ciascuno contiene due componenti principali: il meccanismo di attenzione e la rete feedforward.

Tokenizzazione

La tokenizzazione è il passaggio più elementare e consiste nel suddividere l'input in una serie di token, in modo da associare ogni elemento a un token noto all'interno di un vocabolario predefinito.

Embedding

Una volta tokenizzato l'input, è necessario trasformare gli elementi in numeri. Questo avviene tramite l'embedding, che associa a ogni token un vettore numerico. Se due elementi sono simili, i loro vettori avranno componenti simili; se sono diversi, i vettori saranno differenti.

Codifica posizionale

Dopo aver ottenuto i vettori per ciascun token, il passo successivo è combinare queste informazioni tenendo conto dell'ordine degli elementi. Il metodo più comune è sommare i vettori componente per componente. Tuttavia, la somma è commutativa: ciò significa che sequenze con gli stessi elementi ma in ordine diverso potrebbero generare lo stesso vettore, il che è problematico. Per risolvere questo, si usa la codifica posizionale, che consiste nell'aggiungere a ogni embedding un vettore che rappresenta la posizione dell'elemento nella sequenza. In questo modo, sequenze con gli stessi elementi ma in ordine diverso avranno rappresentazioni distinte.

Blocco Transformer

A questo punto, il modello riceve un vettore per ogni token. Il passo successivo è prevedere l'elemento successivo nella sequenza, compito affidato a una rete neurale molto ampia. Per migliorarne l'efficacia, si introduce un componente chiave: l'attenzione.
Introdotto da A. Vaswani et al. [26], il meccanismo di attenzione consente al modello di considerare il contesto di ogni parola. Ogni blocco Transformer è composto da due elementi principali: il componente di attenzione e il componente feedforward. Questi blocchi vengono concatenati per formare l'intera architettura del Transformer.
Il meccanismo di attenzione affronta il problema del contesto: uno stesso elemento può avere significati diversi a seconda della sequenza in cui è inserito e il modello, senza contesto, non può distinguere tra i due significati.

L'attenzione, invece, consente di avvicinare semanticamente gli elementi correlati: questo arricchisce la rappresentazione vettoriale dell'elemento con informazioni contestuali.
Nei Transformer, si utilizza una forma avanzata chiamata m*ulti-head attention*, che applica più meccanismi di attenzione in parallelo, ciascuno focalizzato su aspetti diversi del contesto. Questo ha migliorato notevolmente la capacità dei modelli di comprendere e generare testo.

Strato softmax

Infine, il modello produce dei punteggi per tutti gli elementi possibili come output. Lo strato softmax trasforma questi punteggi in probabilità, assegnando la probabilità più alta all'elemento più probabile come successivo nella sequenza. Questo processo viene ripetuto elemento dopo elemento, permettendo al Transformer di generare intere sequenze.

2.4.3 Graph Neural Network

Una Graph Neural Network (GNN) [27] appartiene ad una classe di reti neurali artificiali per l'elaborazione di dati che possono essere rappresentati come grafici. In queste strutture, i nodi rappresentano oggetti e i collegamenti le relazioni tra di essi. Le reti GNN funzionano grazie a uno scambio di informazioni tra i nodi del grafo: ogni nodo modifica le informazioni che descrivono il suo stato o il suo contenuto, basandosi sui dati che riceve dai nodi vicini, in modo iterativo. Esistono molte varianti di GNN, che differiscono per il modo in cui avviene questo scambio di informazioni, che viene spesso riadattato basandosi su metodi ricorsivi o convoluzionali.
L'architettura di una GNN generica implementa i seguenti livelli fondamentali [30] [29]:

1. uno strato equivariante di permutazione che mappa una rappresentazione del grafo in una versone aggiornata dello stesso grafo, mantenendo invariata la struttura del grafo stesso. Questo processo viene imple-

mentato attraverso il passaggio di messaggi tra i nodi del grafo. In ogni passaggio, i nodi aggiornano le proprie rappresentazini aggregando i messaggi ricevuti dai loro vicini. Questo permette ad ogni nodo di espandere il proprio campo ricettivo in modo da includere informazioni da un numero maggiore di nodi vicini. A ogni passaggio di messaggi aumenta così la quantità di informazioni a cui ogni nodo ha accesso, migliorando in modo progressivo la comprensione della struttura del grafo;

2. uno strato di pooling locale, che riduce la complessità del grafo tramite sottocampionamento, semplificando la struttura, Questa operazione aumenta il campo ricettivo della rete;

3. un livello di pooling globale, noto anche come livello di lettura, che fornisce una rappresentazione di dimensione fisse dell'intero grafico. Questo strato deve essere invariante alla permutazione, in modo che le permutazioni nell'ordinamento dei nodi e dei bordi del grafico non alterino l'output finale.

Capitolo 3

Analisi di approcci selezionati alla Binary Similarity

In questo capitolo verranno analizzati cinque contributi scientifici rilevanti sul tema della binary similarity: i primi tre basati su modelli di Machine Learning, i due successivi fondati su approcci tradizionali che impiegano tecniche classiche senza ricorrere all'apprendimento automatico.

3.1 Unsupervised Binary Code Translation with Application to Code Similarity Detection and Vulnerability Discovery

Al giorno d'oggi i software vengono spesso compilati per diverse Instruction Set Architectures (ISA), che definiscono il set di istruzioni che una CPU può eseguire. Per molte ISA con risorse limitate di dati (low-resource ISAs) come ARM e, è difficile reperire dataset etichettati sufficienti per addestrare modelli di deep learning.
Per ovviare a questo problema, Iftakhar Ahmad e Lannan Luo nel loro paper *"Unsupervised Binary Code Translation with Application to Code Similarity Detection and Vulnerability Discovery"* [25] propongono di applicare le tec-

niche di Neural Machine Translation (NMT) non supervisionata all'analisi del codice binario.
L'idea fondamentale è che un binario, una volta disassemblato, viene rappresentato come una sequenza di istruzioni in linguaggio assembly. Gli autori propongono quindi di tradurre il codice binario da ISA con poche risorse, come ARM, verso ISA con abbondanti dati disponibili, come x86, permettendo così di utilizzare modelli già addestrati su x86 per analizzare binari di altre architetture.

3.1.1 Metodologia

Partendo dal modello Undreamt [31] viene progettato il modello UnsuperBinTrans che è in grado di tradurre i binari a bassa qualità di ISA in quelli ad alta qualità.
Il modello è mostrato in Figura 3.1.

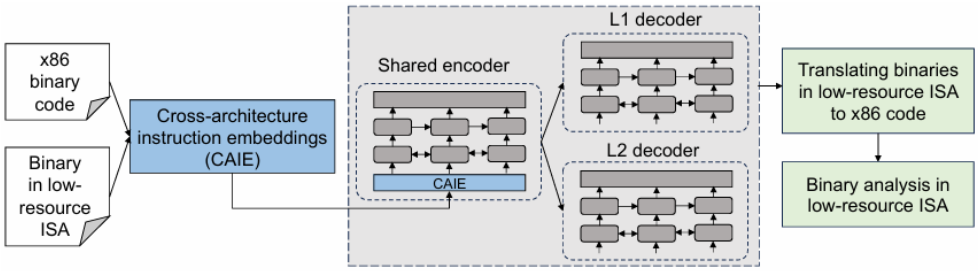

Figura 3.1: Architettura di UnsuperBinTrans [25].

Come prima cosa vengono generati i *cross-architecture instruction embeddings* (CAIE), ovvero le rappresentazioni vettoriali del linguaggio, per due ISA: una ad alta qualità di risorse (x86) e l'altra a bassa qualità di risorse. Successivamente, viene addestrato UnsuperBinTrans. Questo processo utilizza set di dati mono-architecture e, da questi, genera dei CAIE. I dataset mono-architecture per ogni ISA contengono un grande numero di blocchi base presi da quel determinato ISA.

A questo punto, il modello può tradurre un blocco base dall'ISA a bassa qualità di risorse in x86, e il blocco base tradotto condivide una semantica simile a quello iniziale. Attraverso la traduzione è possibile utilizzare un modello istruito su x86 per analizzare il codice tradotto.
Il design di UnsuperBinTrans è stato basato su Undreamt [31], il quale ha ottenuto ottimi risultati nella traduzione grazie a tre principali proprietà della sua architettura:

- dual structure;
- shared encorer;
- fixed embeddings in the encoder.

La dual structure permette di ottenere capacità di traduzione bidirezionale attraverso l'addestramento.
Lo shared encoder produce una rappresentazione dell'input di testo indipendente dalla lingua, perciò i decoder ad essa corrispondenti possono trasformare quella rappresentazione nella lingua richiesta.
Infine, con l'aiuto degli fixed embeddings in the encoder, il modello può facilmente comprendere come comporre rappresentazioni a livello di parola di una lingua per costruire rappresentazioni per frasi più grandi.
Questi embeddings sono pre-addestrati e mantenuti fissi durante l'addestramento, il che rende questa procedura più semplice.
La procedura di addestramento consiste in due componenti principali:

- denoising;
- on-the-fly backtranslation.

Nonostante la proprietà duale di Undreamt lo aiuti a ottenere una conoscenza preliminare della traduzione, alla fine questa procedura si riduce a una semplice copiatura, in cui il modello addestrato potrebbe eseguire le sostituzioni parola per parola tra le lingue di origine e di destinazione. Pertanto, per migliorare questa capacità di traduzione del modello addestrato, viene

introdotto del rumore nei dati di input aggiungendo alcuni scambi casuali di parole contigue.
Successivamente, il modello deve recuperare il testo di input eliminando il rumore dei dati corrotti. Questo processo di aggiunta e rimozione del rumore aiuta il modello a comprendere la reale relazione tra le parole nelle frasi per le lingue che sono date.
UnsuperBinTrans può essere applicato a varie analisi di codice binario, come *code similarity detection* o *vulnerability discovery*.

3.1.2 Risultati ottenuti

Sono stati condotti tre esperimenti per valutare la performance di traduzione di UnsuperBinTrans .
Inizialmente, vengono descritte le impostazioni sperimentali e, successivamente, condotte le analisi quantitative calcolando i punteggi BLEU (Bilingual Evaluation Understudy).
UnsuperBinTrans contiene tre componenti: fastText [15], vecmap [32] e Undreamt [31].
Gli esperimenti sono stati svolti su un computer con Ubuntu 20.04 nel quale vengono settate le dimensioni dell'embedding a 200 e utilizzate le impostazioni predefinite: gli iperparametri, come il tasso di apprendimento, il numero di epoche e la dimensione del batch, sono mantenuti ai loro valori di default.
Viene selezionato un set di pacchetti open-source ampiamente utilizzati, tra cui `binutils` [33], `coreutils` [34], `findutils` [35], `inetutils` [36] e `openssl` [37], per costruire due dataset monoarchitetturali: uno per x86 e l'altro per ARM. Nello specifico, per ogni programma, viene compilato il binario in x86 e ARM.
Sono poi considerati quattro livelli di ottimizzazione: O0, O1, O2 e O3, quando vengono generati i binari. Come risultato, per ogni programma, si ottengono quattro binari per ogni ISA, ognuno dei quali è associato a un determinato livello di ottimizzazione.
Successivamente, i binari vengono disassemblati utilizzando IDA Pro [5] e

vengono estratti i blocchi base. Ogni istruzione in ogni blocco base viene, quindi, processata.
Infine, vengono costruiti quattro dataset monoarchitetturali x86 ed altri quattro ARM. Ognuno di questi contiene un set di blocchi base preprocessati corrispondente ad una determinata ISA e ad un livello di ottimizzazione.
Per quanto riguarda UnsuperBinTrans, dati due dataset corrispondenti a due diverse ISA e lo stesso livello di ottimizzazione, come prima cosa viene utilizzato fastText per generare i *mono-architecture instruction embeddings* (MAIE) per ogni ISA, i quali sono mappati in uno spazio condiviso per generare i CAIE attraverso VECMAP [32].
Successivamente, viene istruito UnsuperBinTrans ad utilizzare i due dataset e i CAIE in modo da tradurre i blocchi base da un'ISA ad un'altra.
Ricordiamo che UnsuperBinTrans è basato sull'apprendimento non supervisionato: non è necessario determinare quale blocco base di quale ISA sia simile ad un altro blocco base di un'altra ISA.
Come prima cosa viene calcolato il BLEU score per misurare la qualità della traduzione, il quale è rappresentato da un numero tra zero e uno. Se lo score è vicino allo zero, la qualità della traduzione è scarsa. Altrimenti, la qualità è buona quando ci si avvicina ad uno.
Per ogni funzione x86, inizialmente si cerca la funzione ARM simile: due funzioni sono dette simili se se sono compilate dallo stesso pezzo di codice sorgente e dissimili se il loro codice sorgente è diverso.
Di conseguenza, per ogni funzione ARM denotata come S, dopo aver identificato la sua funzione x86 simile R, la coppia viene inclusa nel dataset.
Per ogni funzione ARM nel dataset viene utilizzato UnsuperBinTrans per tradurre ciascuno dei suoi blocchi base in x86, ottenendo così una funzione tradotta in x86 denotata come T.
Successivamente, viene calcolato il BLEU score tra T ed R e, infine, il BLEU score medio per tutte le funzioni del dataset.
Si ottengono così i valori 0.76, 0.77, 0.77, 0.76 per i quattro diversi livelli di ottimizzazione, rispettivamente, quando viene utilizzato fastText per la

generazione degli embedding delle istruzioni.
Inoltre, quando viene utilizzato word2vec, si ottiene 0.77, 0.77, 0.77, 0.76 per i quattro livelli di ottimizzazione.
Questi risultati dimostrano che il modello ottiene buone prestazioni di traduzione.

3.1.3 Applicazioni pratiche

Function similarity comparison

Per ogni livello di ottimizzazione, sono casualmente selezionate 7.418 funzioni x86. Per ciascuna funzione x86 selezionata, viene cercata la sua funzione simile e dissimile in ARM, assegnando l'etichetta "1" a ciascuna coppia di funzioni simili e l'etichetta "0" a ciascuna coppia di funzioni dissimili.
Successivamente vengono generati per ciascuna funzione gli embedding di funzione. In particolare in questo lavoro viene utilizzato il seguente approccio:

1. si calcola la frequenza del termine (TF) per ogni istruzione in una funzione;

2. si moltiplica il valore TF per il corrispondente embedding CAIE dell'istruzione:

3. si sommano tutti i CAIE pesati per ottenere l'embedding della funzione.

Per le funzioni ARM, prima vengono tradotte in x86 utilizzando UnsuperBinTrans, e poi è calcolato l'embedding della funzione tradotta con il metodo descritto sopra.
Infine, per misurare la somiglianza tra funzioni, si calcola la similarità cosina tra i loro embedding e poi si determina l'accuratezza del processo di confronto.
L'esperimento, i cui risultati sono mostrati nella Figura 3.2 è stato eseguito in tutti i livelli di ottimizzazione.

Opt. Level	Tool for instruction embedding generation	
	word2vec	fastText
O0	78.09%	81.24%
O1	83.33%	91.11%
O2	87.05%	95.30%
O3	86.76%	94.97%

Figura 3.2: Risultati statistici dei dataset [25].

Poiché UnsuperBinTrans deve prima generare i MAIE e poi eseguire la traduzione, sono stati studiati anche diversi strumenti per generare questi ultimi e valutato quale offre le migliori prestazioni. In particolare, si è fatto uso di fastText e word2vec.
I risultati dimostrano che si sono ottenute accuratezze ragionevoli per ciascun livello di ottimizzazione, tra questi fastText risulta essere il migliore.

3.1.4 Conclusioni

Il deep leaning è stato ampiamente utilizzato per l'analisi del codice binario. Tuttavia, la disponibilità limitata di dati per le ISA a bassa qualità di risorse ostacola l'analisi automatizzata.
In questo lavoro, viene proposto UnsuperBinTrans, un modello che sfrutta la facile reperibilità di dataset per l'architettura x86 per affrontare il problema della scarsità di dati nella ISA meno diffuse.
L'analisi del codice binario ispirata alle tecniche NLP è una direzione promettente per la ricerca, anche se non tutte queste tecniche sono direttamente applicabili a questo ambito.
Questo studio dimostra come le tecniche NLP possano essere sfruttate nell'analisi del codice binario e ne evidenzia l'utilità pratica.

3.2 jTrans: Jump-Aware Transformer for Binary Code Similarity Detection

Recenti studi hanno mostrato che le *Deep Neural Network* (DNN) [28] possono comprendere le istruzioni o i CFG del codice binario e contribuire al rilevamento di similarità dello stesso.
Si differenzia da essi il lavoro proposto da Hao Wang et al. dal titolo *"jTrans: Jump-Aware Transformer for Binary Code Similarity Detection"* [20].
jTrans è la prima soluzione che integra le informazioni di flusso di controllo del codice binario nei modelli linguistici basati su Transformer [26], usando una rappresentazione consapevole dei salti (jump-aware) dei binari analizzati e un nuovo compito di pre-training progettato ad hoc.
Vengono combinati modelli NLP in grado di catturare la semantica delle istruzioni con CFG che catturano le informazioni sul flusso di esecuzione per infierire la rappresentazione del codice. Dal momento che è stato dimostrato che la combinazione di caratteristiche basate su NLP e GNN non produce risultati ottimali, viene proposto per la prima volta di integrare direttamente le informazioni di flusso di controllo nel Transformer.
Il modello Transformer viene modificato per catturare le informazioni sul flusso di controllo condividendo i parametri tra gli embedding dei token e quelli di posizione tra ogni target di salto delle istruzioni.
Inizialmente, vengono utilizzati task di apprendimento non supervisionato per pre-addestrare jTrans, in modo che apprenda la semantica delle istruzioni e le informazioni di flusso di controllo delle funzioni binarie. Successivamente, viene ottimizzato il modello per abbinare funzioni semanticamente simili. In particolare, vengono presi come riferimento i task OM, notando che che questo tipo di problemi possono essere ridotti a problemi di tipo OO impostando la dimensione delle funzioni target ad 1.
In questo modello, inoltre, vengono combinate le caratteristiche dei blocchi base senza fare affidamento sulle GNN.
Insieme a questo approccio innovativo viene presentato un dataset ampio e

diversificato chiamato BinaryCorp, che consente di mitigare la mancanza di diversità che caratterizzano i dataset esistenti.

3.2.1 Metodologia

jTrans è basato sull'architettura Transformer-Encoder e include diverse modifiche significative pensate per renderlo più efficace nell'ambito dell'analisi del codice binario.
La prima modifica all'architettura Transformer è progettata per permettere a jTrans di catturare al meglio le relazioni di salto nel codice, ovvero le informazioni sul flusso di controllo. Per questo, inizialmente, viene preprocessato il codice assembly dell'input binario affinchè includa le relazioni di salto del programma. In secondo luogo, vengono modificati gli embedding dei singoli token in ingresso al Transformer in modo che le istruzioni di origine e destinazione del salto siano semanticamente simili.
La seconda modifica, invece, riguarda l'addestramento del modello proposto. Considerando la somiglianza tra il linguaggio naturale e i linguaggi di programmazione in termini di flusso, viene scelto di utilizzare un metodo di addestramento già consolidato per il Transformer: il *Masked Language Model* (MLM).
L'addestramento basato su questo modello richiede di predire i token mascherati basandosi sul contesto dato dai token vicini, forzando il modello a sviluppare una comprensione contestuale delle relazioni tra le istruzioni.
Inoltre, per incoraggiare il modello ad apprendere la maniera in cui le istruzioni di salto vengono incorporate nel codice, viene proposto un nuovo compito di addestramento che richiede al modello di predire la destinazione di un'istruzione di salto. Questo compito, chiamato *Jump Target Prediction* (JTP), richiede una comprensione approfondita della semantica del codice e contribuisce in modo significativo alle prestazioni del modello.
jTrans è basato sull'architettura BERT [38], che rappresenta lo stato dell'arte tra i modelli linguistici pre-addestrati utilizzati per molti compiti di elaborazione del linguaggio naturale.

In jTrans viene seguito lo stesso approccio utilizzato da BERT per la modellazione dei testi: viene creato un embedding per ogni token e fatto uso del meccanismo di attenzione di BERT per modellare in modo efficace il codice binario.
Tuttavia, i codici binari si differenziano dalle lingue naturali sotto vari aspetti. Per prima cosa, nel codice binario ci sono una quantità eccessiva di vocaboli, in secondo luogo, sono presenti istruzioni di salto. Per ognuna di esse, viene definito come token sorgente l'operando che specifica l'indirizzo della destinazione del salto. Per semplicità, si definisce invece come token destinazione il token dell'istruzione di destinazione, e si rappresenta questa coppia di salto come <*token sorgente, token destinazione*>.
Perciò, è necessario affrontare due problemi al fine di applicare BERT:

- token out-of-vocabulary (OOV): come nell'elaborazione del linguaggio naturale, è necessario addestrare jTrans su un vocabolario di dimensioni fisse che contenga i token più comuni presenti nel corpus analizzato;

- modellazione delle istruzioni di salto: dopo il pre-processing, il codice binario contiene poche informazioni riguardo ai token sorgente e destinazione di una coppia di salto. BERT difficilmente riesce ad inferire la connessione tra essi. Questo problema è aggravato dalla possibile grande distanza tra il token sorgente e quello di destinazione, rendendo l'inferenza contestuale ancora più complessa.

Preprocessing delle istruzioni

Per mitigare il problema dei token OOV viene utilizzato lo strumento di disassemblaggio IDA Pro 7.5 [5] per analizzare i programmi binari in input e ottenere sequenze di istruzioni assembly. Successivamente, vengono applicate le seguenti strategie di tokenizzazione per normalizzare l'assembly e ridurre la dimensione del vocabolario:

1. vengono utilizzati come token i mnemonici e gli operandi;

2. vengono sostituite le stringhe letterali con il token <`str`>;
3. vengono sostituiti i valori costanti con il token <`const`>;
4. vengono mantenuti come token i nomi delle chiamate a funzioni esterne e le etichette, sostituendo invece i nomi delle funzioni chiamate interne con <`function`>;
5. per ogni coppia di salto viene rimpiazzato il token sorgente con un token `JUMP_XXX` dove `XXX` è la posizione del token destinazione della coppia di salto. In questo modo è possibile rimuovere l'impatto degli indirizzi base casuali nei file binari.

Modellazione delle istruzioni di salto

Viene ora affrontata la sfida di rappresentare un'istruzione di salto in una maniera che permetta a jTrans di contestualizzare meglio la natura bipartita, ossia la relazione tra l'origine e la destinazione del salto. Viene scelto di utilizzare codifiche posizionali, che sono parte fondamentale dell'architettura Transformer. Queste codifiche permettono al modello di determinare la distanza tra i token.
La logica implicita di questa rappresentazione è che le distanze maggiori tra i token generalmente indicano una minore influenza reciproca. Tuttavia, le istruzioni di salto legano tra loro delle aree di codice che potrebbero essere molto distanti. Per questo viene modificato il meccanismo di codifica posizionale per riflettere l'effetto di queste istruzioni di salto.
Le modifiche alle codifiche posizionali sono progettate per riflettere il fatto che il token sorgente e destinazione di un'istruzione di salto non sono solo vicini come due token consecutivi, ma hanno anche una forte connessione contestuale.
Questo obiettivo è stato raggiunto attraverso la condivisione di parametri: per ogni coppia di salto, l'embedding del token sorgente viene utilizzato come codifica posizionale del token destinazione. Tale rappresentazione soddisfa due scopi fondamentali: per prima cosa l'embedding condiviso permette al

Transformer di identificare la connessione contestuale tra i token sorgente e destinazione, in secondo luogo, questa forte connessione è mantenuta durante tutto il processo di addestramento dal momento che i parametri condivisi vengono aggiornati simultaneamente per entrambi i token.
Si ipotizza che l'informazione sul flusso di controllo derivante dai salti indiretti possa migliorare notevolmente le prestazioni di jTrans. Tuttavia, riconoscere le destinazioni di tali salti è una sfida ancora aperta ben nota che va oltre l'ambito qui trattato.

Addestramento di jTrans

L'architettura BERT, sulla quale è basato jTrans, utilizza due task di apprendimento non supervisionato per il pre-addestramento.
Il primo task è quello dei MLM, dove BERT ha il compito di ricostruire i token mascherati in modo casuale. Il secondo, invece, è progettato per affinare le capacità contestuali di BERT, richiedendogli di determinare se due frasi sono consecutive.
Il processo di addestramento di jTrans è basato su quello complessivo di BERT, con specifici adattamenti per il dominio: viene preservato il compito MLM, ma viene rimpiazzato il secondo con il JTP. L'obiettivo di quest'ultimo compito è quello di migliorare la comprensione contestuale di jTrans delle istruzioni di salto.
jTrans utilizza, quindi, le stesse procedure di mascheramento di BERT:

- l'80% dei token selezionati casualmente sono rimpiazzati dal token speciale di mascheramento, che indica che devono essere ricostruiti;
- un 10% viene sostituito con altri token scelti casualmente;
- il restante 10% rimane invariato.

Il compito JTP è definito come segue: dato un token sorgente di salto casuale, il modello deve prevedere il token di destinazione corrispondente. Questo compito, difficile persino per degli esseri umani esperti, richiede al modello

di sviluppare una profonda comprensione del CFG. Questo porta ad un miglioramento delle prestazioni di jTrans.
Al termine della fase di pre-addestramento non supervisionato, viene ottimizzato il modello per la BCSD. L'obiettivo è quello di addestrare jTrans in modo che massimizzi la similarità tra coppie di funzioni binarie simili e minimizzi la similarità tra coppie non correlate.

3.2.2 BinaryCorp

Per la BCSD, viene costruito uno specifico dataset basato sui repository ufficiali di ArchLinux [39] e sull'Arch User Repository [40]. Il primo è una distribuzione Linux nota per il suo vasto numero di pacchetti e per i frequenti aggiornamenti, l'Arch User Repository, invece, è gestito dagli utenti e ne conta oltre 77.000, ampliando notevolmente il dataset.
ArchLinux fornisce uno strumento utile chiamato `makepkg`, che consente agli sviluppatori di compilare i pacchetti partendo dal codice sorgente. Questo strumento analizza il file `PKGBUILD`, che contiene le dipendenze e le funzioni di supporto alla compilazione. Dal momento che la BCSD richiede una grande quantità di dati etichettati, viene sfruttata questa infrastruttura per costruire il dataset.
BinaryCorp, il dataset creato per valutare la BCSD su larga scala, consiste in un vasto numero di binari generati da una pipeline di compilazione automatica. Sono stati compilati 48.130 programmi binari con diversi livelli di ottimizzazione e sono state, inoltre, filtrate le funzioni duplicate.
Le statistiche sono riportate nella tabella che segue.

Datasets	# Projects	# Binaries	# Functions
GNUtils	20	100	161,202
Coreutils	115	575	76,174
BinaryCorp-3M Train	1,612	8,357	3,126,367
BinaryCorp-3M Test	364	1,908	444,574
BinaryCorp-26M Train	7,845	38,455	21,085,338
BinaryCorp-26M Test	1,974	9,675	4,791,673

Figura 3.3: Statistiche sul numero di progetti, binari e funzioni nei dataset [20].

La Figura 3.3 mostra chiaramente che BinaryCorp-26M opera su una scala completamente diversa: questo nuovo dataset contiene circa 26 milioni di funzioni, rispetto alle 161.202 di GNUtils [41] e alle 76.174 di coreutils [34]. Pertanto, BinaryCorp-26M è oltre 160 volte più grande di GNUtils e più di 339 volte più grande di coreutils. Questo dataset appena generato offre una base più diversificata e realistica per l'apprendimento e la valutazione. Facendone uso è possibile valutare la scalabilità e l'efficacia di jTrans su una scala nuova e più ampia.

3.2.3 Risultati ottenuti

La valutazione è stata condotta su due dataset: BinaryCorp-3M e BinaryCorp-26M. Inoltre, sono state utilizzate due dimensioni del pool di funzioni, 32 e 10.000, in modo che jTrans e i metodi di riferimento siano valutati in condizioni di difficoltà variabile.
È importante notare che sono stati assegnati casualmente interi progetti ai set di addestramento o test, poichè studi recenti hanno provato che assegnare randomicamente singoli binari può causare perdite di informazioni.

I risultati nelle successive tabelle [3.1] [3.2] [3.3] [3.4] mostrano che jTrans supera tutti i metodi di riferimento con margini significativi.

	MRR							Recall@1						
Models	**O0,O3**	**O1,O3**	**O2,O3**	**O0,Os**	**O1,Os**	**O2,Os**	**Average**	**O0,O3**	**O1,O3**	**O2,O3**	**O0,Os**	**O1,Os**	**O2,Os**	**Average**
Gemini	0.388	0.580	0.750	0.455	0.546	0.614	0.556	0.238	0.457	0.669	0.302	0.414	0.450	0.422
SAFE	0.826	0.917	0.958	0.854	0.927	0.927	0.902	0.729	0.869	0.933	0.766	0.879	0.880	0.843
Asm2Vec	0.479	0.878	0.961	0.536	0.855	0.900	0.768	0.351	0.828	0.942	0.408	0.796	0.863	0.701
GraphEmb	0.602	0.694	0.750	0.632	0.674	0.675	0.671	0.485	0.600	0.678	0.521	0.581	0.584	0.575
OrderMatters-online	0.542	0.740	0.869	0.638	0.702	0.682	0.695	0.414	0.647	0.822	0.515	0.611	0.593	0.591
OrderMatters	0.601	0.838	0.933	0.701	0.812	0.800	0.777	0.450	0.763	0.905	0.566	0.724	0.715	0.687
Genius	0.377	0.587	0.868	0.437	0.600	0.627	0.583	0.243	0.479	0.830	0.298	0.490	0.526	0.478
`jTrans`-Zero	0.594	0.841	0.962	0.649	0.850	0.891	0.797	0.499	0.803	0.945	0.566	0.808	0.853	0.746
`jTrans`	**0.947**	**0.976**	**0.985**	**0.956**	**0.979**	**0.977**	**0.970**	**0.913**	**0.960**	**0.974**	**0.927**	**0.964**	**0.961**	**0.949**

Tabella 3.1: Risultati dei diversi metodi di rilevamento della similarità binaria su BinaryCorp-3M (dimensione del pool = 32) [20].

	MRR							Recall@1						
Models	**O0,O3**	**O1,O3**	**O2,O3**	**O0,Os**	**O1,Os**	**O2,Os**	**Average**	**O0,O3**	**O1,O3**	**O2,O3**	**O0,Os**	**O1,Os**	**O2,Os**	**Average**
Gemini	0.037	0.161	0.416	0.049	0.133	0.195	0.165	0.024	0.122	0.367	0.030	0.099	0.151	0.132
SAFE	0.127	0.345	0.643	0.147	0.321	0.377	0.320	0.068	0.247	0.575	0.079	0.221	0.283	0.246
Asm2Vec	0.072	0.449	0.669	0.083	0.409	0.510	0.366	0.046	0.367	0.589	0.052	0.332	0.426	0.302
GraphEmb	0.087	0.217	0.486	0.110	0.195	0.222	0.219	0.050	0.154	0.447	0.063	0.135	0.166	0.169
OrderMatters	0.062	0.319	0.600	0.075	0.260	0.233	0.263	0.040	0.248	0.535	0.040	0.178	0.158	0.200
Genius	0.041	0.193	0.596	0.049	0.186	0.224	0.214	0.028	0.153	0.538	0.032	0.146	0.180	0.179
`jTrans`-Zero	0.137	0.490	0.693	0.182	0.472	0.510	0.414	0.088	0.412	0.622	0.122	0.393	0.430	0.340
`jTrans`	**0.475**	**0.663**	**0.731**	**0.539**	**0.665**	**0.664**	**0.623**	**0.376**	**0.580**	**0.661**	**0.443**	**0.586**	**0.585**	**0.571**

Tabella 3.2: Risultati dei diversi metodi di rilevamento della similarità binaria su BinaryCorp-3M (dimensione del pool = 10000) [20].

	MRR							Recall@1						
Models	**O0,O3**	**O1,O3**	**O2,O3**	**O0,Os**	**O1,Os**	**O2,Os**	**Average**	**O0,O3**	**O1,O3**	**O2,O3**	**O0,Os**	**O1,Os**	**O2,Os**	**Average**
Gemini	0.402	0.643	0.835	0.469	0.564	0.628	0.590	0.263	0.528	0.768	0.322	0.441	0.518	0.473
SAFE	0.856	0.940	0.970	0.874	0.935	0.934	0.918	0.770	0.902	0.951	0.795	0.891	0.891	0.867
Asm2Vec	0.439	0.847	0.958	0.490	0.788	0.849	0.729	0.314	0.789	0.940	0.362	0.716	0.800	0.654
GraphEmb	0.583	0.681	0.741	0.610	0.637	0.639	0.649	0.465	0.586	0.667	0.499	0.541	0.543	0.550
OrderMatters	0.572	0.820	0.932	0.630	0.692	0.771	0.729	0.417	0.740	0.903	0.481	0.692	0.677	0.652
`jTrans`-Zero	0.632	0.871	0.973	0.687	0.890	0.891	0.824	0.539	0.838	0.961	0.602	0.854	0.853	0.775
`jTrans`	**0.964**	**0.983**	**0.989**	**0.969**	**0.980**	**0.980**	**0.978**	**0.941**	**0.970**	**0.981**	**0.949**	**0.964**	**0.964**	**0.962**

Tabella 3.3: Risultati dei diversi metodi di rilevamento della similarità binaria su BinaryCorp-26M (dimensione del pool = 10000) [20].

	MRR							Recall@1						
Models	**O0,O3**	**O1,O3**	**O2,O3**	**O0,Os**	**O1,Os**	**O2,Os**	**Average**	**O0,O3**	**O1,O3**	**O2,O3**	**O0,Os**	**O1,Os**	**O2,Os**	**Average**
Gemini	0.072	0.189	0.474	0.069	0.147	0.202	0.192	0.058	0.148	0.420	0.051	0.115	0.162	0.159
SAFE	0.198	0.415	0.696	0.197	0.377	0.431	0.386	0.135	0.314	0.634	0.127	0.279	0.343	0.305
Asm2Vec	0.118	0.443	0.703	0.107	0.369	0.480	0.370	0.099	0.376	0.638	0.086	0.307	0.413	0.320
GraphEmb	0.116	0.228	0.498	0.133	0.198	0.224	0.233	0.080	0.171	0.465	0.090	0.145	0.175	0.188
OrderMatters	0.113	0.292	0.682	0.118	0.256	0.295	0.292	0.094	0.222	0.622	0.093	0.195	0.236	0.244
`jTrans`-Zero	0.215	0.570	0.759	0.233	0.571	0.563	0.485	0.167	0.503	0.701	0.175	0.507	0.500	0.426
`jTrans`	**0.584**	**0.734**	**0.792**	**0.627**	**0.709**	**0.710**	**0.693**	**0.499**	**0.668**	**0.736**	**0.550**	**0.648**	**0.648**	**0.625**

Tabella 3.4: Risultati dei diversi metodi di rilevamento della similarità binaria su BinaryCorp-26M (dimensione del pool = 10000) [20].

Per il pool di 32 funzioni jTrans supera il suo concorrente più vicino di 0.07 nella metrica MRR e di oltre 10% nella metrica recall@1.
La differenza di prestazioni diventa più evidente quando si valutano i modelli con il pool di 10.000 funzioni. Per questo setup, jTrans supera il concorrente più vicino di 0.26 nel MRR e di oltre 27% nel recall@1.
I risultati dimostrano i vantaggi di questo approccio proposto, il quale utilizza sia l'architettura Transformer, sia un metodo innovativo per la rappresentazione e l'analisi del CFG. Si riescono a superare significativamente alcuni metodi all'avanguardia, come SAFE [42] che utilizza una rete neurale ricorrente (RNN) e analizza il CFG in modo meno rigoroso, e OrderMatters [43], che utilizza l'architettura Transformer ma analizza ogni blocco in modo indipendente.
Un altro aspetto importante del risultato è il maggiore degrado relativo delle prestazioni dei metodi di riferimento quando la dimensione del pool aumenta.

Effetti della dimensione del pool sulle prestazioni

I risultati appena descritti evidenziano l'effetto della variabile *poolsize*, ovvero la dimensione del pool di funzioni, sulla prestazione degli algoritmi di rilevamento della similarità binaria.
Questi risultati sono particolarmente significativi considerando che studi precedenti utilizzano pool di dimensioni piccole, tra 10 e 200, con alcuni studi che impiegano addirittura un poolsize di 2. Tali configurazioni diventano problematiche dal momento che nelle applicazioni reali la dimensione del pool è

spesso molto più grande, anche di diversi ordini di grandezza.
Per questo motivo viene ora presentata un'analisi degli effetti del poolsize sulle prestazioni dei metodi all'avanguardia (SOTA) per l'analisi binaria.
In particolare, come mostrato nella figura 3.4 sono stati condotti diversi esperimenti con una grande varietà di poolsize -2, 10, 32, 128, 512, 1.000, 10.000- e tracciati i risultati per diverse combinazioni di ottimizzazione.

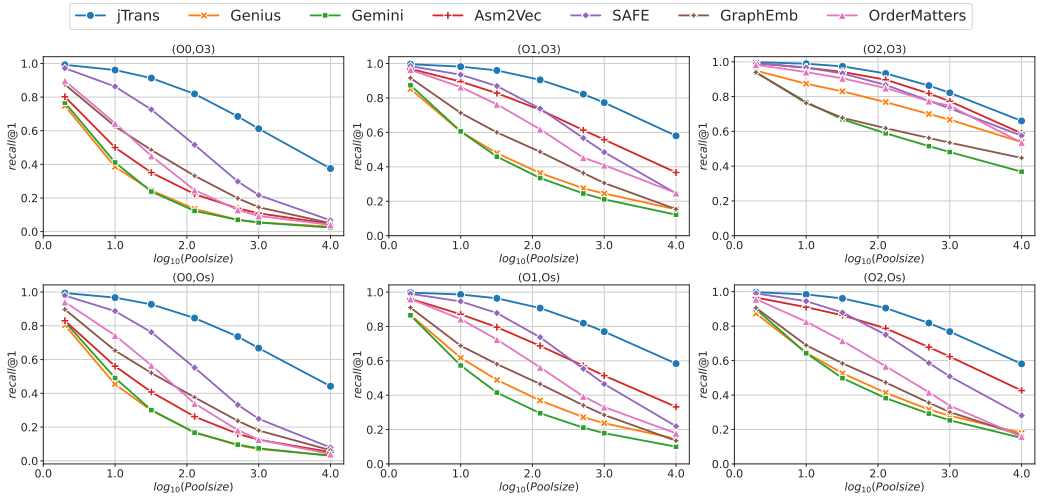

Figura 3.4: Prestazioni dei diversi metodi di rilevamento della similarità binaria su BinaryCorp-3M [20].

I risultati mostrano chiaramente che tutti i metodi di riferimento peggiorano in modo relativo rispetto a jTrans man mano che il poolsize aumenta. Inoltre, questo approccio non mostra cali bruschi nelle prestazioni, al contrario delle prestazioni dei metodi di riferimento che decadono rapidamente una volta raggiunto un poolsize di 100. Questo suggerisce che l'approccio non è influenzato dalla dimensione del pool, quanto dalla maggiore difficoltà del problema di classificazione dovuta all'elevato numero di candidati.
Infine, viene segnalato come per piccoli poolsize, come per esempio 2, le prestazioni dei metodi SOTA come SAFE e Asm2Vec siano quasi identiche a quelle di jTrans, con quest'ultimo che li supera di circa il 2%. Si deduce

che valutare strumenti di analisi binaria su pool di piccole dimensioni non fornisca indicazioni significative sulle loro prestazioni in contesti reali.

3.2.4 Applicazioni pratiche

Ricerca di vulnerabilità nel mondo reale

Il rilevamento delle vulnerabilità è una delle principali applicazioni nella sicurezza informatica, motivo per cui viene valutata la performance di jTrans nel compito reale di ricerca di vulnerabilità su otto *Common Vulnerabilities and Exposures* (CVE) estratti da un dataset di vulnerabilità note. Per ciascuna funzione vulnerabile vengono generate 10 varianti tramite diversi compilatori e livelli di ottimizzazione. La metrica di valutazione è recall@10, che misura la capacità del modello di recuperare le vulnerabilità rilevanti entro i primi dieci risultati: più alto è il valore, maggiore è la copertura dei casi correttamente identificati. Per simulare uno scenario realistico sono state utilizzate tutte le funzioni del progetto come pool di ricerca, con un numero variabile tra 3.038 e 60.159. I risultati nella figura 3.5 mostrano il confronto con i due metodi di riferimento principali: SAFE e Asm2Vec.

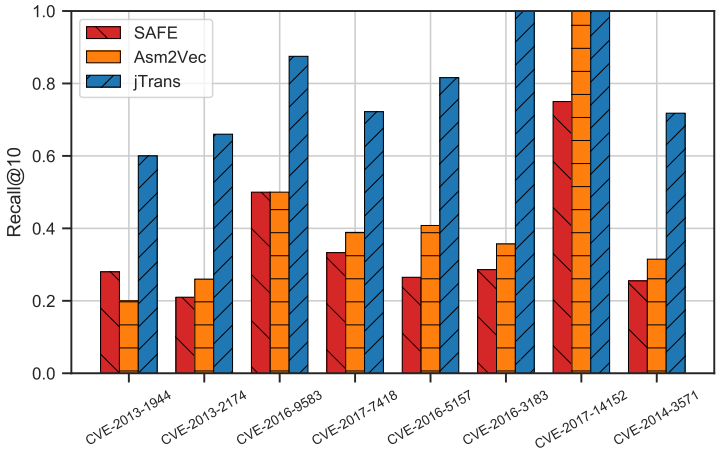

Figura 3.5: Recall@10 nella ricerca di vulnerabilità nel mondo reale [20].

È chiaro che, per la maggior parte dei CVE, le prestazioni di jTrans sono significativamente superiori rispetto a quelle degli altri due metodi.
Questi risultati dimostrano che jTrans può essere impiegato in modo efficace come strumento di ricerca di vulnerabilità in scenari reali, grazie alle sue capacità di mantenere alte le prestazioni anche con pool di grandi dimensioni.

3.2.5 Conclusioni

In questo studio ci si è concentrati sull'addestramento di jTrans sulla sola architettura x86, ma la tecnica proposta può essere applicata anche ad altre. jTrans rappresenta una soluzione innovativa per la BCSD, superando le prestazioni degli approcci all'avanguardia. Può essere applicato in numerosi ambiti, tra cui:

- rilevamento di vulnerabilità note in binari sconosciuti;
- individuazione e clustering di malware;
- rilevamento di plagio software;
- analisi di patch;
- analisi della supply chain del software.

Un esempio concreto è il mondo dell'IoT, nel quale il riutilizzo del codice è estremamente comune per lo sviluppo. Soluzioni come jTrans possono contribuire a rilevare vulnerabilità presenti in librerie open source utilizzate nei dispositivi.
Nello scenario delle blockchain e smart contracts il riutilizzo del codice è diffuso, ma i rischi di sicurezza sono elevati. Potrebbe, quindi, essere impiegato jTrans per rilevare vulnerabilità in questi ambienti, dove il problema è reso ancora più critico dalla dipendenza tra progetti.

3.3 Binary Similarity Detection using Machine Learning

Trovare procedure simili in binari *stripped*, ossia privi di informazioni non essenziali, è un problema rilevante in vari contesti di sicurezza informatica. Molti studi precedenti hanno proposto soluzioni che privilegiano l'accuratezza a scapito della velocità o che si concentrano su casi meno complessi, lasciando aperta la sfida di coniugare precisione ed efficienza. Un approccio particolarmente interessante a questo problema è stato presentato nel 2018 da Noam Shalev e Nimrod Partush in *"Binary Similarity Detection using Machine Learning"* [44], che hanno introdotto il modello Zeek e il metodo proc2vec.
L'idea centrale è quella di sfruttare il Machine Learning insieme al principio della *somiglianza per composizione*: il codice viene suddiviso in frammenti più piccoli, trasformati in rappresentazioni vettoriali e confrontati tramite predittori addestrati a riconoscere procedure semanticamente equivalenti. Per supportare questo processo, viene introdotto proc2vec, che traduce ogni procedura in segmenti canonici e li incorpora in uno spazio vettoriale, permettendo poi a una rete neurale di stimare la similarità.
Il database usato per l'addestramento è costituito da decine di milioni di esempi generati compilando diversi progetti open source con vari compilatori, versioni, livelli di ottimizzazione e architetture. I test, condotti su software non incluso nel set di addestramento, mostrano che Zeek riesce a mantenere un'elevata accuratezza ed è significativamente più veloce rispetto alle tecniche precedenti, rendendo praticabile l'analisi di carichi di lavoro reali.

3.3.1 Metodologia

Il tool Zeek realizzato all'interno di questo lavoro adotta il concetto di *strands* come elemento fondamentale dell'algoritmo. Uno strand è il set di istruzioni di un blocco di codice necessarie per calcolare il valore di una determinata variabile.

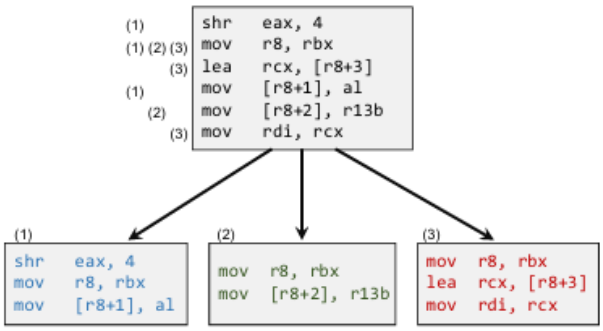

Figura 3.6: Decomposizione in strands [44].

La Figura 3.6 mostra un esempio di decomposizione di un blocco di codice nei suoi strands che lo costituiscono. Si noti che una singola istruzione può essere associata a più strands all'interno di una sezione di codice. Ad esempio, la seconda istruzione nel blocco di codice della figura appartiene a tutti e tre gli strands che compongono la sezione di codice. È importante notare che due strands sintatticamente diversi possono essere equivalenti, e che uno strand non è necessariamente sintatticamente contiguo.

Proc2vec

L'intuizione alla base dell'algoritmo proc2vec si fonda sul principio della similarità per composizione, secondo il quale due segnali sono simili se è facile comporre uno dei due utilizzando ampi frammenti contigui dell'altro. Questo principio si è dimostrato efficace nel rilevamento della similarità, soprattutto nell'analisi dei programmi. Pertanto, in questo lavoro si vogliono utilizzare gli strands che compongono una sezione di codice come insieme di caratteristiche. Questi ultimi vengono trasformati in numeri e assemblati per formare un vettore che rappresenta il codice corrispondente. Per apprendere la frequenza e la co-occorrenza degli strands, viene addestrato un modello di apprendimento automatico. Di conseguenza, le sezioni di codice che contengono strands rari o rare combinazioni di essi, tendono ad essere più uniche e a ricevere un punteggio di similarità più alto. Al contrario, combinazioni comuni di strands tendono ad avere l'effetto opposto. Ad esempio, gli strands

introdotti dal compilatore, come le operazioni di gestione dello stack, sono molto frequenti e irrilevanti ai fini della similarità.

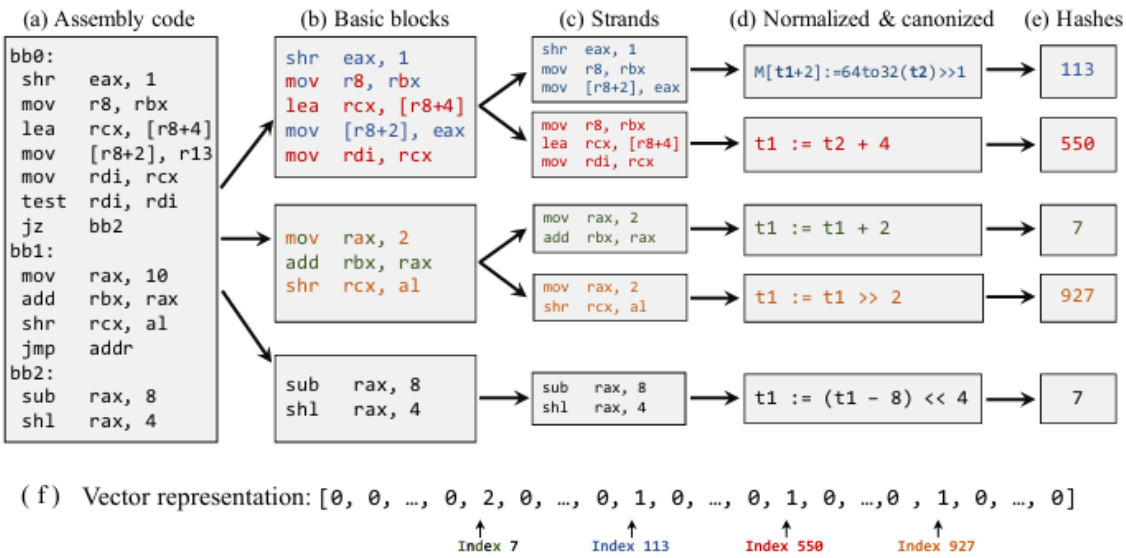

Figura 3.7: Fasi di trasformazione da procedura a vettore [44].

Come descritto nella Figura 3.7, dato il codice assembly di una procedura, la tecnica di trasformazione in vettore si articola in cinque passaggi:

1. viene suddivisa la procedura in blocchi base. Un blocco base è definito in base alla posizione dell'istruzione `jmp` nel codice binario della procedura;

2. ogni blocco base viene ulteriormente scomposto in strand. Nella Figura 3.7b, ogni strand è rappresentato con un colore diverso. Le istruzioni possono appartenere a più strand contemporaneamente;

3. successivamente, vengono portati gli strand sintatticamente diversi ma semanticamente equivalenti alla stessa rappresentazione testuale. Per fare ciò, vengono utilizzate tecniche introdotte in altri lavori e vengono ottimizzati e canonizzati tutti gli strand. Come mostrato in Figura

3.7d, questo step modifica la rappresentazione degli strand in una forma canonica: somme successive vengono raggruppate, moltiplicazioni per potenze di due vengono convertite in shift, e le operazioni aritmetiche vengono riordinate in modo coerente;

4. viene applicato un hashing MD5 a b-bit sulla rappresentazione testuale degli strand, traducendo ciascuno in un intero compreso nell'intervallo $\{0, 2^b - 1\}$. Se b è piccolo, è probabile che che si verifichino collisioni, ovvero che strand diversi vengano mappati allo stesso valore hash;

5. infine, come mostrato in Figura 3.7f, viene utilizzato l'insieme di interi ottenuti come indici per costruire un vettore di lunghezza 2^b , dove ogni elemento del vettore rappresenta il numero di volte in cui l'indice corrispondente appare nell'insieme. Gli elementi del vettore possono quindi essere maggiori di uno, e la somma totale degli elementi corrisponde al numero di strand contenuti nella procedura.

Per implementare questo algoritmo, viene utilizzata la libreria open-source PyVEX [45], in particolare il suo binary lifter per convertire il codice assembly in VEX-IR e suddividerlo in strand (step 2). Inoltre, viene sfruttata l'ottimizzatore VEX per normalizzare ogni strand (step 3).

Generazione dei dati

Per generare i dati vengono presi vari progetti open-source reperiti in rete e compilati per diverse architetture utilizzando diversi tipi di compilatori, versioni e livelli di ottimizzazione. Successivamente, viene applicato l'algoritmo proc2vec su ciascuna delle procedure binarie risultanti, in modo da creare una lista di vettori, ognuno dei quali rappresenta una procedura in una specifica configurazione di compilazione. Impilando tutti questi vettori viene costruita la matrice M.
Utilizzando la conoscenza preliminare sull'origine di ciascun vettore, viene costruita una *match-list*. Questa lista contiene coppie di indici che rappresentano righe corrispondenti in M. In altre parole, se la coppia (i, j) appare

nella match list, allora i vettori M_i (i-esima riga di M) e M_j provengono dallo stesso codice sorgente o dalla medesima procedura. Inoltre, per insegnare al classificatore che ogni procedura è simile a sé stessa e che l'ordine di input di una coppia non ha importanza, viene estesa ogni coppia corrispondente (x, y) in un gruppo di quattro coppie: $(x, y), (x, x), (y, y), (y, x)$. Queste coppie vengono etichettate con 1 (match) e usate per generare un dataset con etichette positive.
Per generare esempi negativi, invece, vengono create coppie casuali di vettori che provengono da procedure non equivalenti. In questo modo, nessuna delle coppie compare nella match list; esse vengono etichettate con 0, ovvero non-match, e utilizzate come dataset di esempi negativi.
Una somiglianza tra sezioni di codice è un evento relativamente raro e questo problema viene affrontato in due modi:

1. utilizzando la metrica CROC [46], progettata per valutare problemi di classificazione sbilanciati al fine di stimare le prestazioni del classificatore;

2. generando un dataset fortemente sbilanciato ed esaminando i diversi rapporti tra il numero di esempi etichettati negativamente e positivamente. Dopo aver esaminato vari rapporti, viene impostato il rapporto a 6.

Infine, viene costruito il dataset completo mescolando l'unione dei dataset etichettati positivamente e negativamente. Complessivamente, si ottiene un database con circa venti milioni di esempi.

Progettazione di un Classificatore a Rete Neurale

Per stimare il punteggio di similarità tra una coppia di vettori, viene costruito un classificatore basato su deep learning utilizzando TensorFlow [47]. Il modello è composto da un layer di input, due layer nascosti completamente connessi e un layer di output softmax, come mostrato in Figura 3.8.

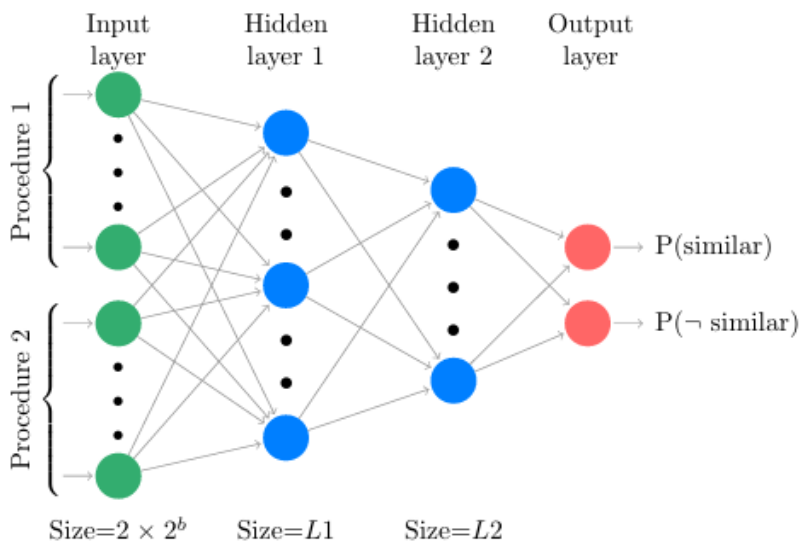

Figura 3.8: Scheletro delle rete neurale [44].

Il layer di input è composto da 2x2^b neuroni, corrispondenti ai due vettori che rappresentano le due procedure di input. Il primo e il secondo layer nascosto hanno dimensioni $L1$ e $L2$, rispettivamente. Infine, l'output è una funzione softmax a 2 neuroni che rappresenta la probabilità di similarità tra le sezioni di codice che hanno generato i vettori di input.
Viene allenato il modello utilizzando la funzione di costo dell'entropia incrociata, una regolarizzazione dropout di 0.1, una dimensione batch pari a 32 e 3 epoche di addestramento.

3.3.2 Risultati ottenuti

Il sistema Zeek viene valutato confrontandolo con GitZ [48]. Vengono testate diverse dimensioni di hash, da 4 a 14 bit, per trovare il miglior compromesso tra accuratezza e prestazioni. Il risultato ottimale risulta essere quello con 10 bit, che corrisponde a vettori di lunghezza 1024.
Questa configurazione fornisce il miglior punteggio di accuratezza nei test su progetti come `coreutils` [34]
Con la configurazione ottimale (hash a 10 bit, $L1$=512, $L2$=128), Zeek viene

testato su vari progetti open source.
Il modello ottiene CROC scores superiori a GitZ in tutti i test.
Questa rete neurale è in grado di generare circa 7.000 predizioni al secondo per core, contro le 25 di GitZ: questo significa che Zeek è oltre 250 volte più veloce. Inoltre, in termini di latenza, possiede una pipeline più snella con una latenza inferiore del 15% rispetto a GitZ.

3.3.3 Conclusioni

In questo paper viene presentato un approccio per la BCSD basato sul principio della similarità per composizione, affiancato da tecniche di apprendimento automatico. Viene ideato proc2vec, un algoritmo per rappresentare sezioni di codice sotto forma di vettori, senza ricorrere a processi di estrazione di caratteristiche progettati manualmente. Viene dimostrato che questo approccio raggiunge un'elevata accuratezza e un'elevata velocità di elaborazione, risultando pratico per l'utilizzo in scenari reali.

3.4 Statistical Similarity of Binaries

Tra i diversi approcci proposti in letteratura per affrontare il problema della somiglianza binaria, non tutti si basano su tecniche di Machine Learning. Alcuni lavori, infatti, adottano ancora metodi più tradizionali di tipo statistico e composizionale. In questa categoria rientra lo studio pubblicato nel 2016 da Yaniv David et al. dal titolo *"Statistical Similarity of Binaries"* [49], che rappresenta uno dei contributi più significativi in tale direzione.
In questo paper viene affrontato il problema dell'individuazione di procedure simili all'interno di binari *stripped* e presentato un nuovo approccio statistico per misurare la somiglianza tra due procedure. La definizione di somiglianza qui presentata consente di rilevare codice simile anche quando è stato compilato con compilatori diversi o modificato.
L'idea principale è l'utilizzo della somiglianza per composizione: scomporre il codice in frammenti più piccoli e confrontabili, definire la somiglianza

semantica tra i frammenti e applicare un ragionamento statistico per elevare la simiglianza tra frammenti a una somiglianza tra procedure. Questo approccio viene implementato in uno strumento chiamato Esh, che viene utilizzato per individuare diverse vulnerabilità rilevanti attraverso compilatori e versioni differenti, tra cui Heartbleed (CVE-2014-0160), Shellshock (CVE-2014-6271 e CVE-2014-7169) e Venom (CVE-2015-3456). Viene dimostrato che Esh produce risultati ad alta accuratezza, generando pochi o nessun falso positivo.
Data una procedura di query q e una vasta collezione T di procedure destinazione in forma binaria, l'obiettivo è definire quantitativamente la somiglianza di ciascuna procedura $t \in T$ rispetto alla query q.
La sfida principale è, quindi, quella di definire una nozione semantica di somiglianza che sia sufficientemente precisa da evitare falsi positivi, ma anche abbastanza flessibile da consentire l'individuazione del codice in qualsiasi combinazione dei seguenti scenari:

- il codice è stato compilato utilizzando versioni diverse dello stesso compilatore;
- il codice è stato compilato utilizzando compilatori di fornitori diversi;
- è stata compilata una versione diversa del codice.

É necessario un metodo che possa operare senza alcuna informazione sul codice sorgente e/o sulla toolchain utilizzata nella creazione dei binari, basandosi esclusivamente sulle procedure in forma binaria.

3.4.1 Metodologia utilizzata

Viene presentata una nuova nozione di somiglianza tra procedure, basata sui seguenti elementi chiave:

- decomposizione della procedura in strands: le procedure vengono suddivise in strands che risultano più semplici da confrontare;

- confronto tra strands: con un verificatore di programmi si controlla se due strands sono semanticamente equivalenti, assumento l'equivalenza degli input e verificando se i valori intermedi e finali coincidono. Quando non sono equivalenti viene definita una nozione quantitativa di somiglianza tra strands basata sulla proporzione di valori corrispondenti rispetto al numero totale di valori nel strand;

- ragionamento statistico sugli strands: viene proposto un framework statistico per analizzare la somiglianza tra intere procedure a partire dalla somiglianza tra strands. Viene calcolato un punteggio globale di evidenza di somiglianza tra procedure sommando i punteggi locali di evidenza tra strands.

Similarità per composizione

Due procedure vengono identificate come probabilmente simili se strands non banali di una possono essere utilizzati per comporre l'altra, permettendo alcune trasformazioni legate al compilatore o a patch.
La procedura viene decomposta in blocchi base, i quali sono ulteriormente divisi in strands utilizzando una tecnica di slicing.
La similarità tra strands viene verificata con l'aiuto di un verificatore di programmi che fornisce la flessibilità di determinare la similarità tra due strands, cioè che siano (parzialmente) equivalenti in input-output, anche se producono il risultato utilizzando istruzioni diverse, allocazioni di registri differenti o un ordine di programma diverso.
Inoltre, per valutare la similarità delle procedure è stato definito un *Global Evidence Score*, $GES(q|t)$, che rappresenta la probabilità che una procedura di query q sia simile ad una procedura target t. Questo punteggio globale si basa sulla composizione di più *Local Evidence Score*, $LES(sq|t)$, punteggi locali che rappresentano la somma delle probabilità che ciascun strand sq in q abbia uno strand simile st in t.

Decomposizione in strands ed evidenze statistiche

Le procedure vengono decomposte in strands e, dati due set di strands ottenuti da due procedure da confrontare, si assume l'esistenza di un meccanismo per calcolare una misura di similarità tra due strands basata sulla proporzione di variabili di output che coincidono quando vengono forniti input equivalenti. Questa metrica di similarità viene denotata come *Variable Correspondence Proportion* (VCP).
Si assume che il valore VCP tra due strands, s_q e s_t, sia noto, e lo si trasforma in una misura probabilistica di similarità tra strands, indicata come $P_r(s_q|s_t)$. Successivamente, viene calcolata $P_r(sq, H_0)$, ovvero la probabilità che un dato strand sq corrisponda a un processo casuale. Utilizzando questi valori, viene calcolato il *Likelihood Ratio* ($LR(sq)$), ovvero il rapporto di somiglianza, dal quale viene derivato il valore GES.

Local e global evidence scores

Dopo aver presentato la decomposizione delle procedure q e t in strands e aver definito il rapporto di somiglianza per ciascun strand sq e target t, è possibile calcolare il LES. Questo punteggio locale riflette il livello di confidenza che sq abbia un equivalente semantico non banale all'interno di t. Il GES tra q e t è semplicemente la somma di tutti i valori LES(sqt) ottenuti dopo la decomposizione di q in strand, e rappresenta il livello di confidenza che q possa essere composto da parti non banali di t e sia quindi effettivamente simile ad essa.

Similarity semantics

A questo punto viene definito formalmente il concetto di similarità semantica tra strands. Uno strand è modellato come un insieme di tracce di esecuzione, ossia sequenze di stati che rappresentano i valori assunti dalle variabili durante il flusso del programma. Per confrontare due strands, viene introdotta la nozione di *corrispondenza di variabili*, una funzione che associa

variabili del primo strand a quelle del secondo.
Se, sotto una determinata corrispondenza, dati input equivalenti, gli stati finali delle due tracce risultano coerenti, i due strands sono considerati semanticamente equivalenti. Tuttavia, l'obiettivo non è limitarsi a distinguere equivalenza o non equivalenza, ma graduare la similarità. Per questo motivo viene introdotta la misura *Variable Containment Proportion* (VCP), che quantifica la proporzione di variabili di uno strand che trovano un corrispettivo valido nell'altro. Il valore di VCP varia da 0, nessuna corrispondenza, a 1, equivalenza completa, consentendo così di catturare anche somiglianze parziali.

Encoding Similarity as a Program Verifier Query

Dopo aver definito la nozione astratta di similarità, il paper affronta la questione della sua computazione concreta. Viene proposto di trasformare il confronto tra due strands in una query per un program verifier. A tale scopo, entrambi i frammenti vengono tradotti in Boogie IVL [50], un linguaggio intermedio adatto alla verifica formale. Il processo prevede i seguenti passi:

1. traduzione dei due strands in procedure Boogie [50];

2. creazione di assunzioni sugli input, che ipotizzano uguaglianza tra variabili corrispondenti;

3. composizione sequenziale delle istruzioni dei due strands, per rappresentarne la semantica in un unico programma;

4. aggiunta di asserzioni sugli output, per verificare che le variabili finali corrispondenti abbiano valori equivalenti.

Il program verifier, nel caso specifico Boogie, controlla se tali asserzioni siano sempre valide per tutte le possibili esecuzioni. Se lo sono, i due strands risultano equivalenti. Se non lo sono, viene comunque calcolato un valore di VCP, che quantifica il grado di similarità parziale.

3.4.2 Risultati ottenuti

L'idea centrale è testare se, dato un frammento di codice vulnerabile noto, sia possibile individuare automaticamente altre istanze della stessa vulnerabilità in un ampio corpus di procedure compilate in modo eterogeneo.
L'approccio viene quindi validato con esperimenti che simulano scenari reali di ricerca di vulnerabilità: un binario noto e vulnerabile viene utilizzato come query, e l'algoritmo deve recuperare altre procedure affette dalla stessa debolezza, distinguendole correttamente da codice non correlato.
É stato realizzato un prototipo chiamato Esh, che accetta in input un procedimento di query e un database di procedure in formato binario:

- ogni eseguibile è stato suddiviso in singole procedure con IDA Pro [5] e successivamente tradotto in una rappresentazione intermedia;
- questo codice è stato poi tradotto in Boogie IVL [50], così da poter essere processato dal program verifier;
- infine, da ciascuna procedura sono stati estratti gli strands su cui viene applicato il calcolo della similarità semantica.

Per mettere alla prova l'approccio in uno scenario realistico, il test è stato costruito includendo otto pacchetti software reali e vulnerabili, ciascuno associato a una CVE nota, come ad esempio Heartbleed (CVE-2014-0160), Shellshock (CVE-2014-6271 e CVE-2014-7169), Venom (CVE-2015-3456), e altre vulnerabilità critiche.
Per ciascun pacchetto sono stati compilati diversi eseguibili con più compilatori e versioni differenti, includendo anche varianti patchate. Nel complesso il corpus conteneva circa 1500 procedure in formato binario, prive di informazioni di debug.
Sono state individuate tre dimensioni principali del problema:

- differenti versioni dello stesso compilatore: ad esempio, la stessa procedura compilata con gcc 4.6, 4.8 e 4.9;

- compilatori diversi: come gcc, Clang e icc, che producono binari sintatticamente molto differenti;
- codice patchato: versioni vulnerabili confrontate con versioni corrette, per valutare se la similarità semantica resista anche a modifiche del codice sorgente.

Metodologia di valutazione

Valutare un metodo che produce un punteggio quantitativo di similarità non è banale. Non esiste una soglia unica in grado di separare sempre veri positivi da falsi positivi. Per questo motivo vengono utilizzati strumenti di valutazione noti nell'apprendimento automatico:

- ROC (Receiver Operating Characteristic): misura l'accuratezza di un classificatore binario considerando tutte le possibili soglie;
- CROC (Concentrated ROC): variante della ROC pensata per scenari in cui i veri positivi sono rari e costosi da validare manualmente. Penalizza in modo più severo i falsi positivi, riflettendo meglio il contesto della ricerca di vulnerabilità.

Gli esperimenti mostrano che Esh è in grado di:

- riconoscere procedure vulnerabili compilate con compilatori diversi e versioni diverse;
- distinguere in maniera netta tra codice correlato e non correlato, anche quando le differenze sintattiche sono ampie;
- mantenere alte prestazioni in scenari di codice patchato, dove gli approcci puramente sintattici tendono a fallire.

In generale, la valutazione mostra valori di ROC e CROC molto vicini a 1.0 per la maggior parte delle vulnerabilità, indicando un'elevata precisione e un bassissimo tasso di falsi positivi. Gli autori osservano che questa accuratezza

deriva proprio dall'uso della similarità semantica basata sugli strands, che consente di superare i limiti dei metodi precedenti basati su confronti puramente sintattici.
Gli esperimenti condotti mostrano che il metodo proposto, implementato nello strumento Esh, è in grado di individuare in maniera accurata procedure simili anche quando compilate con compilatori differenti, versioni diverse o sottoposte a patch.

#	Alias/Method	CVE/Stats	#BB	#Strands	S-VCP		S-LOG		Esh	
					FP	ROC/CROC	FP	ROC/CROC	FP	ROC/CROC
1	Heartbleed	2014-0160	15	92	107	0.967 / 0.814	0	1.000 / 1.000	0	1.000 / 1.000
2	Shellshock	2014-6271	136	430	246	0.866 / 0.452	3	0.999 / 0.995	3	0.999 / 0.996
3	Venom	2015-3456	13	67	0	1.000 / 1.000	1	1.000 / 1.000	0	1.000 / 1.000
4	Clobberin' Time	2014-9295	41	233	351	0.797 / 0.343	65	0.987 / 0.924	19	0.993 / 0.956
5	Shellshock	2014-7169	88	294	175	0.889 / 0.541	40	0.987 / 0.920	0	1.000 / 1.000
6	ws-snmp	2011-0444	6	98	42	0.981 / 0.879	5	0.999 / 0.990	1	1.000 / 0.997
7	wget	2014-4877	94	181	332	0.885 / 0.600	11	0.998 / 0.988	0	1.000 / 1.000
8	ffmpeg	2015-6826	11	87	222	0.9212 / 0.6589	97	0.9808 / 0.8954	0	1.000 / 1.000

Tabella 3.5: Valori CROC, ROC e falsi positivi [49].

Nella Tabella 3.5 vengono mostrati i valori di ROC e CROC ottenuti, i quali si avvicinano a 1.0 nella maggior parte dei casi, evidenziando un'elevata precisione e un tasso trascurabile di falsi positivi.
Un risultato particolarmente rilevante è che la similarità semantica, basata sulla decomposizione in strands e sul confronto tramite program verifier, si dimostra molto più robusta rispetto agli approcci puramente sintattici. Questo consente di superare problemi classici come la dipendenza dal compilatore, la variazione di ottimizzazioni e le modifiche introdotte da patch di sicurezza.

3.4.3 Conclusioni

In sintesi, i risultati sperimentali confermano che il metodo basato su similarità semantica e composizione di strands è altamente efficace nel riconoscere codice simile tra binari. La capacità di operare correttamente in presenza di compilatori diversi, versioni multiple e patch costituisce un progresso significativo rispetto ai metodi precedenti. Tuttavia, rimangono alcune limitazioni

legate ai costi computazionali e ai casi di procedure troppo semplici, che richiedono ulteriori ottimizzazioni e studi futuri.

3.5 Cross-Architecture bugs search in binary executables

L'ultimo dei due lavori presi in esame che non fa uso di tecniche di Machine Learning è *"Cross-Architecture bugs search in binary executables"* [51] pubblicato nel 2017 da Yannik Pewny et al. che introduce un approccio per la ricerca di bug nei binari in un contesto cross-architecture, ossia indipendente dall'ISA.
In questo paper, viene proposto un sistema per derivare firme di bug a partire da vulnerabilità note. La procedura si basa sul calcolare degli hash semantici dei blocchi base del binario, i quali vengono successivamente utilizzati per individuare parti di codice che si comportano in modo simile alla firma del bug, rivelando così le sezioni che contengono la vulnerabilità. Come risultato, è possibile identificare vulnerabilità in codice binario affetto da bug, compilato per qualsiasi delle architetture supportate.
Viene, quindi, compiuto il primo passo verso l'identificazione delle vulnerabilità nel software binario e su più architetture. Una volta che un bug è noto si mira a identificare parti ugualmente vulnerabili in altri binari che potrebbero essere stati compilati per architetture diverse.
Viene utilizzata una firma di bug che rappresenta una funzione vulnerabile per trovare istanze simili dello stesso bug.
L'obiettivo è assistere un analista umano in questi scenari, dove egli definisce una firma di bug una sola volta e poi la utilizza per cercare parti simili in altri binari software, qualsiasi sia l'architettura. Per fare ciò viene proposto un meccanismo che utilizza una rappresentazione unificata dei binari, in modo da poterli confrontare tra diverse architetture. Il codice binario di qualsiasi architettura viene convertito verso una rappresentazione intermedia, a partire dalla quale viene colta la semantica del binario a livello di blocco base.

Per ognuno di essi sono costruite delle formule di assegnazione che descrivono il comportamento del blocco in termini di variabili di input e output. Una variabile di input è qualsiasi elemento che influenzi una variabile di output, come registri della CPU o contenuti della memoria.
Le variabili di input vengono campionate casualmente in modo da monitorare i loro effetti su quelle di output. Questa analisi produce una lista di coppie input/output (I/O) per ogni formula di assegnazione, che cattura la semantica effettiva del blocco base.
Sebbene la sintassi del codice simile possa variare tra le diverse architetture CPU, è possibile utilizzare queste semantiche per confrontare blocchi base di diverse ISA.
Viene utilizzata la rappresentazione semantica per cercare la firma del bug in altri software arbitrari che potrebbero essere vulnerabili al bug definito nella firma. La firma del bug può essere derivata automaticamente da un programma binario noto come vulnerabile oppure dal codice sorgente, e può semplicemente rappresentare l'intera funzione vulnerabile. Per preservare le prestazioni viene utilizzato MinHash [52] per trovare corrispondenze adeguate tra blocchi base. Una volta trovate, viene proposto un algoritmo che sfrutta i CFG per espandere la ricerca all'intera firma del bug. Come output, questo sistema elenca le funzioni ordinate in base alla loro somiglianza con la firma del bug.

3.5.1 Metodologia

Si parte dal presupposto che codice simile derivi spesso da codice condiviso e leggermente modificato, e che quindi possa contenere lo stesso bug, potenzialmente critico per la sicurezza.
Grazie all'utilizzo di una firma di bug, ovvero un frammento di codice binario che rappresenta un'istanza specifica di una classe di vulnerabilità, vengono cercati possibili vulnerabilità in un altro programma binario, detto target. La metodologia inizia proprio con la derivazione di questa firma, che succes-

sivamente viene trasformata insieme al codice del programma target in una rappresentazione intermedia.

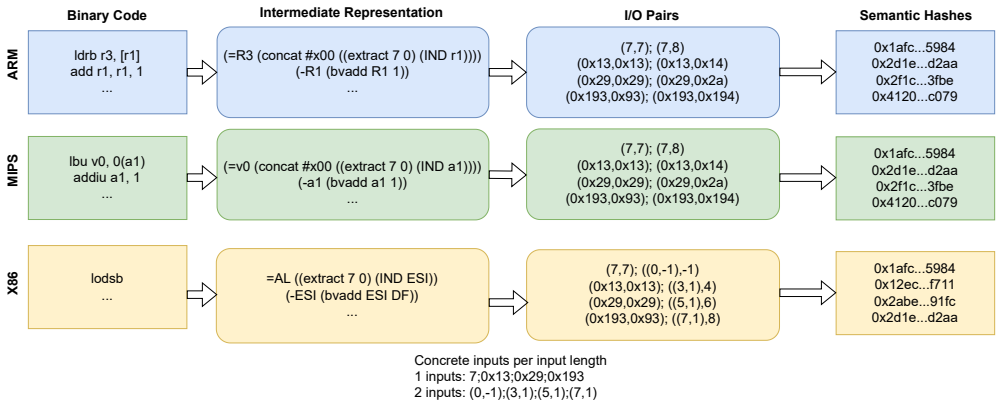

Figura 3.9: Fase di trasformazione.

La Figura 3.9 mostra infatti come il codice assembly, sia della firma del bug che del programma target, venga convertito in una rappresentazione intermedia che produce una lista di formule di assegnazione. Successivamente, utilizzando valori di input concreti e casuali, viene campionato il comportamento input/output delle formule. Nell'ultima fase, vengono costruiti degli hash semantici compatti a livello di blocco base basati sulle coppie I/O, che permettono di confrontare in modo efficiente il comportamento dei blocchi base.

Durante la fase di ricerca, viene utilizzata la firma del bug trasformata per identificare vulnerabilità nei binari target anch'essi trasformati. Si cercano corrispondenze per ciascun blocco base della firma all'interno del programma target. Per ogni coppia, viene applicato un algoritmo di espansione guidato dal CFG, che opera localmente. L'algoritmo amplia la corrispondenza iniziale includendo ulteriori blocchi base sia dalla firma che dal programma target, e calcola la somiglianza tra la firma del bug e il programma target, restituendo una lista di posizioni nel codice ordinate per similarità rispetto alla firma.

Firme di bug

Una firma di bug è simile al normale codice binario: è composta da blocchi base e dalle possibili transizioni di flusso di controllo tra questi blocchi. Di conseguenza, qualsiasi sezione di blocchi base può essere utilizzata come firma di bug.
Sebbene l'obiettivo sia cercare vulnerabilità nei binari, quindi senza accesso al codice sorgente, è possibile utilizzare informazioni aggiuntive, come i dati di debug, per individuare automaticamente i blocchi base corrispondenti alla parte vulnerabile della funzione, ottenendo così firme di bug con uno sforzo manuale minimo.
Tuttavia, è difficile stimare in modo generale come la dimensione della firma influenzi i risultati: un blocco base non caratteristico, ma essenziale per la struttura della vulnerabilità e discriminante nel suo contesto, può migliorare significativamente i risultati. Al contrario, un blocco base caratteristico ma non essenziale per la vulnerabilità potrebbe generare falsi positivi.

Unificare set di istruzioni tra architetture

È evidente che i set di istruzioni di architetture come x86, ARM e MIPS sono piuttosto differenti. Non solo variano le convenzioni di chiamata e gli accessi alla memoria, ma anche la complessità e la lunghezza generale del codice assembly. Per questo motivo, un passaggio fondamentale di questo approccio consiste nell'unificare il codice binario proveniente da diverse architetture.
Viene utilizzato un disassemblatore standard per estrarre la struttura del codice binario, come le funzioni, i blocchi base e i CFG. Successivamente, le istruzioni complesse vengono trasformate in istruzioni semplici, di tipo RISC, e unificate. Questo processo consente di astrarre dagli elementi specifici dell'architettura e facilita la normalizzazione simbolica.
Il set di istruzioni unificato ci consente di confrontare sintatticamente le singole istruzioni binarie. Il passo successivo consiste nell'estrarre la semantica del codice binario. Per farlo, vengono aggregati i passaggi computazionali re-

lativi a ciascuna variabile di output di un blocco base, ottenendo così formule di assegnazione precise per ogni registro di output o locazione di memoria.
É stato riscontrato che l'estrazione delle formule per blocco base funziona efficacemente in scenari con architetture diverse, compilatori differenti e vari livelli di ottimizzazione.
A questo punto, si dispone di descrizioni precise dell'effetto di ciascun blocco base sullo stato del programma. Tuttavia, queste descrizioni sono ancora puramente sintattiche. Per poter individuare bug, viene adottata una metrica che misura la somiglianza tra frammenti di codice. Essa viene costruita basandosi sul campionamento. In primo luogo, vengono generati valori casuali e concreti, utilizzati come input per le formule di ciascun blocco base. Queste valutazioni puntuali catturano la semantica della computazione eseguita. La somiglianza tra codici viene quindi espressa assumendo che frammenti di codice simili condividano un numero maggiore di coppie input/output (I/O) rispetto a quelli dissimili.
Maggiore è il numero di coppie I/O condivise e uguali tra due blocchi base, maggiore sarà la loro similarità. Tuttavia, confrontare direttamente tutte queste coppie I/O risulterebbe molto costoso dal punto di vista computazionale. Per questo, viene adottata una tecnica basata su *hash localmente sensibili* (LSH).
Un hash LSH ha la proprietà che la similarità tra gli hash riflette la similarità tra gli oggetti da cui sono stati generati. In particolare, si è scelto di utilizzare MinHash, che soddisfa le proprietà degli LSH e converge verso l'indice di somiglianza.
Grazie a questo approccio, confrontare due blocchi base si riduce semplicemente al confronto tra i loro hash semantici.

Confronto di strutture binarie più ampie

A questo punto, si è in grado di individuare coppie di blocchi base simili, che rappresentano potenziali candidati per una corrispondenza estesa alla firma del bug. Tuttavia, per confrontare un'intera firma di bug, il confronto

tra singoli blocchi base non è sufficiente.
Per prima cosa, viene selezionato un blocco base dalla firma del bug e confrontato con tutti i blocchi base del programma da analizzare. Successivamente, tramite un algoritmo chiamato *Best Hit Broadening* (BHB), si espande la corrispondenza iniziale lungo i CFG sia della firma del bug che del programma target. BHB cerca di massimizzare localmente la similarità senza tornare indietro, ma in maniera ottimale a livello locale, fino a coprire l'intera firma del bug. Questo processo viene ripetuto per ciascun blocco base della firma, producendo infine una lista di funzioni ordinate in base alla loro similarità con la firma del bug.

3.5.2 Risultati ottenuti

É stata condotta una valutazione sistematica per capire come la scelta del compilatore e il livello di ottimizzazione influenzano la precisione dell'algoritmo. Sono stati selezionati i programmi più grandi del pacchetto `coreutils` e confrontati tra loro: tutti i 12 binari sono stati analizzati in coppie, per un totale di 144 combinazioni, utilizzando un confronto funzione per funzione.
Per quanto riguarda il confronto tra architetture, invece, è stato condotto un ulteriore esperimento nel quale sono state confrontate tutte le funzioni di un binario A con tutte le funzioni di un altro binario B. Idealmente, ogni funzione dovrebbe comparire al primo posto nella classifica di similarità tra funzioni.
I risultati si avvicinano molto all'ideale nel caso di binari appartenenti alla stessa architettura, e mostrano un grado di similarità sufficiente anche tra architetture diverse nella maggior parte dei casi.

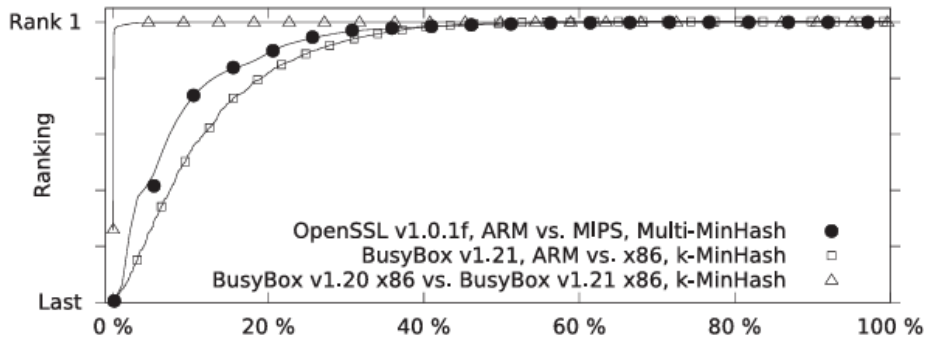

Figura 3.10: Ranking dei risultati [51].

La Figura 3.10 illustra il ranking dei risultati di similarità tra funzioni binarie ottenuti mediante diversi algoritmi di hashing, confrontando diverse versioni di software e architetture.
L'asse X rappresenta la percentuale cumulativa di query analizzate, mentre l'asse Y indica la posizione del risultato corretto nella classifica.
Le curve mostrano come Multi-MinHash posizioni rapidamente i risultati corretti nei primi rank anche tra architetture differenti, indicando un'elevata accuratezza di retrieval. Il metodo k-MinHash, nel confronto tra architetture diverse, mostra una progressione più lenta, suggerendo una minore capacità di identificare immediatamente i match corretti.
Nel confronto tra versioni della stessa architettura, k-MinHash raggiunge rapidamente Rank 1 per la maggior parte delle query, confermando l'efficacia dell'algoritmo in scenari omogenei dal punto di vista architetturale. In generale, curve più ripide e prossime al Rank 1 indicano una maggiore capacità del metodo di posizionare correttamente i risultati simili nelle prime posizioni della classifica.

3.5.3 Conclusioni

É stato dimostrato che il confronto semantico tra codici binari è possibile tra diverse architetture CPU, anche sotto ipotesi realistiche. Questo rappresenta un progresso rispetto alle ricerche precedenti, che si limitavano al

confronto del codice binario su una singola architettura. Questa metrica innovativa consente un confronto al dettaglio del codice, che è stata applicata con successo per identificare vulnerabilità reali in software closed-source.

Capitolo 4

Confronto tra gli approcci analizzati

In questo capitolo vengono analizzati i vantaggi e i limiti dei cinque lavori esaminati nel Capitolo 3. Vengono messi a confronto i tre approcci basati su tecniche di Machine Learning e i due metodi tradizionali, per poi discutere le differenze e le possibili complementarità tra le due tipologie di soluzioni.

4.1 Confronto tra metodi basati su Machine Learning

Gli algoritmi basati su Machine Learning, le cui caratteristiche sono riassunte nella Tabella 4.1, rappresentano gli approcci più avanzati nell'ambito della binary similarity detection.

Metodo	**Unsupervised Binary Code Translation [25]**	**jTrans: Jump-Aware Transformer [20]**	**Binary Similarity Detection using Machine Learning [44]**

Approccio	Traduzione non supervisionata tra architetture	Transformer jump aware	Machine Learning tradizionale
Caratteristiche distintive	Utilizza autoencoder e mapping tra codici di architetture diverse	Modella i salti nel flusso di controllo	Usa estrazione di feature e classificatori
Vantaggi	Non richiede etichette, utile per cross-architecture	Elevata precisione, ottimo per codici con logica complessa	Semplice da implementare, interpretabile
Applicazioni principali	Cross-architecture function similarity e vulnerability discovery su ISA con pochi dati	Rilevamento di similarità semantica tra funzioni e analisi di flusso di controllo	Embedding semantico per clustering, malware detection e matching di funzioni note
Limiti	Complesso da addestrare, meno preciso senza supervisione	Richiede training supervisionato e dataset ben etichettati	Meno efficace su codici offuscati o cross-architecture

Tabella 4.1: Confronto tra approcci basati su Machine Learning.

Ogni metodo è stato analizzato in base al suo approccio principale, alle caratteristiche distintive, ai vantaggi e ai limiti, con l'obiettivo di evidenziarne le differenze principali.
Iftakhar Ahmad e Lannan Luo [25] propongono un approccio non supervi-

sionato con applicazioni sia al rilevamento di similarità tra procedure sia alla scoperta di vulnerabilità in grado di ottenere buoni risultati. Tuttavia, risulta più difficile da addestrare e meno preciso senza supervisione.
Per quanto riguarda il lavoro di Hao Wang et al. [20], gli autori utilizzano un differente approccio basato sull'architettura Transformer, che consegue ottimi risultati in termini di accuratezza, a fronte però di un maggiore costo computazionale, della necessità di dataset ben etichettati e di un calo di accuratezza all'aumentare della dimensione del pool di funzioni.
Infine, Noam Shalev e Nimrod Partush [44] implementano un modello basato su Machine Learning tradizionale che risulta più semplice e interpretabili, ma soffre limitazioni nella gestione di scenari complessi come l'analisi cross-architecture o la presenza di tecniche di offuscamento. Vi sono, inoltre, alcune limitazioni strutturali: la capacità di analisi si basa su log generati da traffico di rete, il che implica una dipendenza da ambienti controllati o dataset preesistenti per garantire la riproducibilità degli esperimenti.

4.2 Confronto tra metodi tradizionali

Nel contesto della BCSD, i metodi tradizionali si distinguono principalmente per il livello di granularità, la precisione nell'individuazione di vulnerabilità e la scalabilità. I due approcci principali sono basati sulla similarità statistica e sull'analisi semantica cross-architecture. La Tabella 4.2 ne sintetizza le caratteristiche principali.

Caratteristica	**Statistical Similarity of Binaries [49]**	**Cross-Architecture Bug Search in Binary Executables [51]**
Tipo di confronto	Statistico, basato su feature globali	Semantico, basato su comportamento I/O

Unità di analisi	Intero binario o funzioni	blocchi base
Architetture supportate	Singola architettura	Multi-architettura
Tecnica principale	Hashing di caratteristiche binarie	Hashing semantico con MinHash
Precisione nel localizzare vulnerabilità	Bassa, utile per clustering	Alta, utile per ricerca di bug
Robustezza contro ottimizzazioni	Limitata	Elevata
Scalabilità	Alta, veloce	Più lenta, ma parallelizzabile
Applicazioni principali	Clustering, classificazione	Ricerca di vulnerabilità
Verifica autonoma della vulnerabilità	Non prevista	Non automatica, ma supportata da firma semantica

Tabella 4.2: Confronto tra approcci cross-architecture.

Dal confronto emerge chiaramente come l'approccio statistico proposto da Yaniv David et al. [49] sia più veloce e scalabile, ma meno preciso nella localizzazione di vulnerabilità specifiche. Inoltre, quando il codice analizzato è molto breve o composto da frammenti banali, la misura di similarità perde significato, poiché i frammenti possono coincidere casualmente. Al contrario, l'approccio semantico proposto nello studio di Yannik Pewny et al. [51] si distingue per la capacità di confrontare codice binario a livello di blocchi base e CFG, anche tra diverse architetture CPU. Questo metodo, sebbene computazionalmente più oneroso, consente una localizzazione fine delle vulnerabilità e una maggiore robustezza rispetto alle ottimizzazioni del

compilatore. Il confronto tra software a livello binario risulta, però, complesso, poiché la rappresentazione del codice dipende fortemente dall'ambiente di compilazione. Nonostante ciò, molte tecniche di ottimizzazione del compilatore sono già gestite dal metodo. Un'altra criticità riguarda le prestazioni: il metodo risulta piuttosto lento, sebbene gran parte dei calcoli possa essere parallelizzata, rendendo l'approccio scalabile. Inoltre, le operazioni più intensive dal punto di vista computazionale vengono eseguite una sola volta, riducendo l'impatto sulle fasi successive di analisi. Nel complesso, pur con queste limitazioni, l'approccio dimostra una buona capacità di affrontare la sfida della ricerca di vulnerabilità su più architetture.
In sintesi, i due metodi presentano vantaggi complementari: il primo adatto a scenari su larga scala e per compiti di clustering, il secondo preferibile per la ricerca accurata di vulnerabilità cross-architecture.

4.3 Confronto tra approcci ML e metodi tradizionali

Negli ultimi anni, la BCSD ha subito una profonda trasformazione metodologica. I primi approcci, di tipo tradizionale, si basavano su tecniche statiche e semantiche come l'analisi dei CFG e il matching simbolico, offrendo soluzioni interpretabili ma spesso fragili rispetto alle ottimizzazioni del compilatore e poco scalabili.
Al contrario, i metodi più recenti fondati sul Machine Learning, come Zeek [44] e jTrans [20], hanno introdotto modelli capaci di apprendere rappresentazioni semantiche del codice binario, sfruttando reti neurali e architetture Transformer. In particolare, jTrans si distingue per l'integrazione esplicita dell'informazione di flusso di controllo all'interno del modello linguistico, superando i limiti dei precedenti approcci basati su GNN e ottenendo risultati significativamente superiori in scenari reali. Questo passaggio dagli approcci basati su regole a quelli basati su apprendimento automatico rappresenta un cambiamento significativo, con implicazioni rilevanti in termini di accuratez-

za, scalabilità e applicabilità pratica.
La Tabella 4.3 sintetizza le principali differenze tra gli approcci tradizionali e quelli basati su Machine Learning analizzati in questa tesi.

Caratteristica	**Machine Learning [25] [20] [44]**	**Tradizionali [49] [51]**
Robustezza e ottimizzazioni	Alta	Bassa
Scalabilità	Elevata	Limitata
Interpretabilità	Bassa	Alta
Accuratezza	Moderata	Alta
Dipendenza da caratteristiche progettate manualmente	Bassa	Alta (caratteristiche apprese automaticamente)
Applicabilità a scenari reali	Limitata	Estesa (vulnerabilità, malware, plagio)

Tabella 4.3: Confronto tra approcci basati su Machine Learning e approcci tradizionali.

Si evidenzia un netto divario in termini di efficacia e applicabilità tra gli approcci tradizionali e i modelli basati su Machine Learning per la BCSD.
In primo luogo, la robustezza rispetto alle ottimizzazioni del compilatore è un limite evidente per le soluzioni classiche, che si basano principalmente sui CFG facilmente alterabili. Al contrario, modelli come jTrans dimostrano una resilienza elevata, grazie alla capacità di apprendere rappresentazioni semantiche profonde che integrano anche il flusso di controllo.
Sul piano della scalabilità, gli approcci tradizionali soffrono di costi computazionali elevati e non sono adatti a dataset di grandi dimensioni, mentre le soluzioni neurali, in particolare quelle Transformer-based, gestiscono milioni

di funzioni con efficienza.
L'interpretabilità rappresenta un punto di forza dei metodi classici, che permettono di seguire il ragionamento passo-passo; tuttavia, i modelli deep learning, pur essendo meno trasparenti, offrono una precisione superiore.
Un altro vantaggio dei modelli neurali è la ridotta dipendenza da caratteristiche manuali, invece i metodi tradizionali richiedono ingegnerizzazione esperta, i modelli come jTrans apprendono automaticamente le caratteristiche rilevanti dai dati.
Infine, l'applicabilità a scenari reali è limitata per le soluzioni classiche, spesso testate su dataset sintetici e poco vari; al contrario, gli altri modelli sono spesso validati su un corpus ampio e realistico, dimostrando efficacia in compiti concreti come la ricerca di vulnerabilità, la rilevazione di malware e il plagio software.

Capitolo 5

Conclusioni e sviluppi futuri

Nel corso di questo lavoro è stata condotta un'analisi comparativa approfondita tra approcci tradizionali e metodi basati su Machine Learning per la rilevazione della similarità tra codici binari.
L'indagine ha permesso di evidenziare come le tecniche classiche, pur offrendo solidità e interpretabilità, risultino spesso limitate nella capacità di generalizzazione e nella scalabilità, soprattutto in contesti cross-architecture. Al contrario, i modelli basati su apprendimento automatico si sono dimostrati particolarmente efficaci nel cogliere relazioni semantiche profonde tra frammenti di codice, anche in presenza di architetture differenti. Tuttavia, questi approcci presentano anch'essi delle criticità, legate principalmente alla dipendenza da dataset ampi e ben etichettati, non sempre disponibili per ambienti con risorse limitate.
Guardando al futuro, si aprono diverse prospettive di ricerca. Una direzione promettente consiste nello studio delle unità di rappresentazione più efficaci per il codice binario: se trattare le istruzioni come parole si è rivelato utile, resta da esplorare il potenziale di alternative come i byte grezzi o la combinazione di opcode e operandi. Comprendere se la scelta della rappresentazione debba essere modulata in base all'ambiente di esecuzione o al tipo di analisi potrebbe portare a modelli più adattivi e precisi. Inoltre, l'estensione di questi approcci a compiti diversi dalla similarità e dalla scoperta di vulne-

rabilità potrebbe ampliare significativamente il loro impatto. Un'altra sfida aperta riguarda il bilanciamento tra accuratezza e efficienza computazionale, soprattutto in scenari reali con grandi volumi di dati. Lo sviluppo di modelli capaci di confrontare direttamente coppie di funzioni binarie, pur mantenendo tempi di risposta compatibili con applicazioni operative, rappresenta un obiettivo ambizioso ma cruciale. Infine, l'integrazione con strumenti di analisi statistica e la capacità di correlare eventi semantici tra binari e metadati di rete potrebbe aprire nuove frontiere nell'automazione del rilevamento e nell'attribuzione del codice.

In sintesi, la ricerca sulla binary similarity si configura come un ambito dinamico e multidisciplinare, in cui l'innovazione metodologica e la comprensione semantica del codice si fondono per affrontare sfide sempre più complesse. I risultati ottenuti in questa tesi non solo confermano la validità degli approcci analizzati, ma delineano con chiarezza le traiettorie future lungo cui il campo potrà evolvere.

Bibliografia

[1] B. S. Baker, U. Manber, and R. Muth, "Compressing differences of executable code," 1999.

[2] C. Irfan, Ul Haq anc Juan, "Asurvey of binary code similarity," 2021.

[3] Z. Wang, K. Pierce, and S. McFarling, "Bmat - a binary matching tool," 1999.

[4] G. Inc., "Bindiff," https://www.zynamics.com/bindiff.html, n.d.

[5] Hex-Rays, "Ida pro disassembler and debugger," https://hex-rays.com/ida-pro/, n.d.

[6] K. Christopher, K. Engin, M. Darren, R. William, and V. Giovanni, "Polymorphic worm detection using structural information of executables," in *Proceedings of the International Symposium on Recent Advances in Intrusion Detection (RAID)*, 2005.

[7] D. Gao, M. K. Reiter, and D. Song, "Binhunt: Automatically finding semantic differences in binary programs," in *International Conference on Information Security and Cryptology (ICISC)*, 2008.

[8] X. Hu, T. Chiueh, and K. G. Shin, "Large-scale malware indexing using function-call graphs," in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2009.

[9] W. Khoo, A. Mycroft, and A. W. Roscoe, "Rendezvous: A search engine for binary code," in *Proceedings of the 10th Working Conference on Mining Software Repositories (MSR)*. IEEE Press, 2013, pp. 329–338.

[10] J. Jang, A. Agrawal, and D. Brumley, "Redebug: Finding unpatched code clones in entire OS distributions," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2012, pp. 48–62.

[11] Y. David and E. Yahav, "Tracelet-based code search in executables," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2014.

[12] J. Pewny, B. Garmany, R. Gawlik, C. Rossow, and T. Holz, "Cross-architecture bug search in binary executables," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2015.

[13] N. Lageman, E. D. Kilmer, R. J. Walls, and P. D. McDaniel, "Bindnn: Resilient function matching using deep learning," in *Proceedings of the International Conference on Security and Privacy in Communication Systems (SecureComm)*, 2016.

[14] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 26, 2013, pp. 3111–3119.

[15] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *Transactions of the Association for Computational Linguistics*, vol. 5, pp. 135–146, 2017.

[16] Y. David, U. Alon, and E. Yahav, "Neural graph embedding for neural network based binary code similarity detection," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 2017, pp. 69–84.

[17] B. Liu, Y. Huo, and Z. Xu, "Cross-version binary code similarity detection with dnn," in *Proceedings of the 33rd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2018, pp. 129–139.

[18] S. H. H. Ding, B. C. M. Fung, and P. Charland, "Asm2vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization." IEEE, 2019.

[19] Y. Duan, X. Li, J. Wang, and H. Yin, "Deepbindiff: Learning program-wide code representations for binary diffing," in *Proceedings of the 27th Annual Network and Distributed System Security Symposium (NDSS 2020)*, 2020. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2020/02/24311-paper.pdf

[20] H. Wang, W. Qu, G. Katz, W. Zhu, Z. Gao, H. Qiu, J. Zhuge, and C. Zhang, "jtrans: Jump-aware transformer for binary code similarity detection," *arXiv preprint arXiv:2205.12713v1*, 2022.

[21] F. Artuso, "Deep learning based binary code analysis," Ph.D. dissertation, Facoltà di Ingegneria dell'Informazione, Informatica e Statistica, Ph.D. Program in Engineering in Computer Science, Università di Bologna, 2025.

[22] M. D. Hill, "Proprietary versus open instruction sets," *IEEE Micro*, 2016.

[23] Red Hat, Inc., "Arm vs x86 architectures," https://www.redhat.com/it/topics/linux/ARM-vs-x86, 2024.

[24] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools, 2nd Edition*. Addison-Wesley, 2006.

[25] A. Iftakhar and L. Lannan, "Unsupervised binary code translation with application to code similarity detection and vulnerability discovery," 2024, p. 2.

[26] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2017.

[27] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: https://arxiv.org/abs/1609.02907

[28] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: https://www.deeplearningbook.org/

[29] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, 2021.

[30] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1263–1272. [Online]. Available: https://arxiv.org/abs/1704.01212

[31] M. Artetxe, G. Labaka, E. Agirre, and K. Cho, "Unsupervised neural machine translation," in *International Conference on Learning Representations (ICLR)*, 2018.

[32] M. Artetxe, G. Labaka, and E. Agirre, "A robust self-learning method for fully unsupervised cross-lingual mappings of word embeddings," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018, pp. 789–798. [Online]. Available: https://arxiv.org/abs/1804.07755

[33] "Gnu binutils," https://www.gnu.org/software/binutils/.

[34] "Gnu core utilities," https://www.gnu.org/software/coreutils/, accessed: 22-09-2025.

[35] "Gnu find utilities," https://www.gnu.org/software/findutils/.

[36] "Gnu inetutils," https://www.gnu.org/software/inetutils/.

[37] "Openssl: Open source toolkit for ssl/tls," https://www.openssl.org/.

[38] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[39] "Arch linux," https://www.archlinux.org/.

[40] "Arch user repository (aur)," https://aur.archlinux.org/.

[41] "Gnu utilities," https://www.gnu.org/software/gnutils/, accessed: 22-09-2025.

[42] L. Massarelli, G. A. Di Luna, F. Petroni, L. Querzoni, and R. Baldoni, "Safe: Self-attentive function embeddings for binary similarity," *arXiv preprint arXiv:1811.05296*, 2018. [Online]. Available: https://arxiv.org/abs/1811.05296

[43] Z. Yu and Y. Cao, "Order matters: Semantic-aware neural networks for binary code similarity detection," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, 2020, pp. 5767–5774. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/5466

[44] S. Noam and P. Nimrod, "Binary similarity detection using machine learning," 2018.

[45] angr Project, "Pyvex: Vex ir lifter for binary analysis," https://github.com/angr/pyvex.

[46] S. J. Swamidass, "A croc stronger than roc: measuring, visualizing and optimizing early retrieval," *Bioinformatics*, vol. 26, no. 3, pp. 441–447, 2010.

[47] M. Abadi, A. Agarwal, P. Barham *et al.*, "Tensorflow: Large-scale machine learning on heterogeneous systems," https://www.tensorflow.org/, 2015.

[48] Y. David, N. Partush, and E. Yahav, "Similarity of binaries through re-optimization," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017)*. New York, NY, USA: ACM, 2017, pp. 79–94.

[49] D. Yaniv, P. Nimrod, and Y. Eran, "Statistical similarity of binaries," 2016.

[50] M. Barnett, B.-Y. E. Chang, R. DeLine, B. Jacobs, and K. R. M. Leino, "Boogie: A modular reusable verifier for object-oriented programs," in *International Symposium on Formal Methods for Components and Objects (FMCO)*.

[51] J. Pewny, B. Garmany, R. Gawlik, C. Rossow, and T. Holz, "Cross-architecture bug search in binary executables," *it – Information Technology*, 2017.

[52] A. Z. Broder, "On the resemblance and containment of documents," in *Proceedings of the Compression and Complexity of Sequences 1997*. IEEE, 1997.